Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

PROJECT REPORT ON

"KutumbSetu"

Submitted in parallel Fulfilment of Requirements

Diploma in Information Technology
By

Rishi Soni: 57508240006

Rishik Jariwala: 57508240026

Nirmay Patel: 57508240027

Under the Guidance of

Mr. Pankaj D Rathod

Lecturer, Information Technology Department

Academic Year: 2026-2027

Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

PROJECT REPORT ON

"KutumbSetu"

Submitted in parallel Fulfilment of Requirements

Diploma in Information Technology
By

Rishi Soni: 57508240006

Rishik Jariwala: 57508240026

Nirmay Patel: 57508240027

Under the Guidance of

Mr. Pankaj D Rathod

Lecturer, Information Technology Department

Project Guide **Head** **Principal**

Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

CERTIFICATE

This is to certify that the project entitled **"KutumbSetu"** is a bonafide work of:

Rishi Soni: 57508240006

Rishik Jariwala: 57508240026

Nirmay Patel: 57508240027

Submitted in partial fulfillment of the requirement for the award of Diploma in Information Technology.

Project Guide **Head** **Principal**

Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

Approval Sheet

This is to certify that the project entitled **"KutumbSetu"** is a bonafide work of:

Rishi Soni: 57508240006

Rishik Jariwala: 57508240026

Nirmay Patel: 57508240027

Submitted in partial fulfillment of the requirement for the award of Diploma in Information Technology.

Signature:

Name : _____

Internal Examiner

Signature:

Name : _____

External Examiner

Date:

Place: Mumbai

Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

INFORMATION TECHNOLOGY DEPARTMENT

VISION

To create a learning environment that nurtures students and transform them into competent IT Diploma Graduates.

MISSION

M1: To impart technical and managerial skills for pursuing academic excellence through dynamic learning environment.

M2: To foster industry ready graduates by acquiring and utilizing latest technology.

M3: To strengthen holistic development and Professionalism in the Diploma graduates.

PROGRAMME EDUCATIONAL OBJECTIVES

After successful completion of the program students will be able to:
PEO1: Imbibe core knowledge and utilise associated technologies to provide domain related solutions.

PEO2: Be capable of adapting to the rapid pace of technological dynamics through professional competence.

PEO3: Develop a holistic personality equipped with leadership qualities, team skills and humane approach towards society.

PROGRAM SPECIFIC OUTCOMES

PSO1: Students will demonstrate fundamental knowledge in core domains of IT such as Software Development, Databases and Information Systems

PSO2: Students will acquire skills that can provide IT solutions in the field of Networking, IOT, Machine Learning and Cloud Computing.

Shri Vile Parle Kelavani Mandal's

Shri Bhagubhai Mafatlal Polytechnic and College of Engineering

Vile Parle-(West), Mumbai-400056

Table of Contents

Abstract..... 6

Chapter 1..... 7 - 9

1.1 Introduction 7

1.2 Problem Statement 8

1.3 Proposed Solution 9

Chapter 2..... 10 - 14

2.1 System Requirements..... 10

2.2 Tools and Technologies Used 11

2.3 Features 14

2.4 Gantt Chart..... 15

Chapter 3..... 16 - 20

3.1 System Design..... 16

3.1.1 Flowchart..... 16

3.1.2 DFD Level 0..... 17

3.1.3 DFD Level 1..... 17

3.1.4 UMLUse Case Diagram..... 18

3.1.5 UML Activity Diagram..... 19

3.1.6 UML Component Diagram..... 20

3.1.7 Deployment Diagram..... 20

Chapter 4..... 21 - 87

4.1 Literature Review..... 21

4.2 Source Code 23

4.3 Outputs 75

Chapter 5..... 88 - 94

5.1 Advantages..... 88

5.2 Future Scope..... 90

5.3 Limitations..... 91

5.4 Acknowledgement 92

5.5 Conclusion..... 93

5.6 References..... 94

Abstract

KutumbSetu is a secure community management platform designed to digitally transform the administration of community organizations, trusts, and social groups. Many communities still rely on paper registers, spreadsheets, and manual processes to manage donations, member records, receipts, announcements, and family information, resulting in inefficiency, data redundancy, and lack of transparency.

The proposed system provides a centralized mobile application where administrators can manage donor records, generate digital receipts, maintain member directories, publish community updates, visualize donation analytics, and represent family relationships through an interactive family tree. The application also includes role-based access control to ensure secure and authorized data management. By digitizing community operations, **KutumbSetu** aims to improve transparency, simplify administration, strengthen communication, and provide an organized platform for community members.

Chapter 1

1.1 Introduction

The **Samast Darji Samaj Babariyawad** is a community organization representing the Hindu Darji (tailor) community originally tracing their regional roots or lineages to Babariyawad, a historic region located in the Kathiawar peninsula of Gujarat. Like many broader Gujarati Darji groups, the members historically claim Rajput or Kshatriya ancestry, later adopting textile crafting, tailoring, and intricate embroidery as their primary livelihood. Today, the community has a significant diaspora, with active social welfare chapters like the Samast Darji Samaj Babariyavad Mumbai that work to unite families through cultural and various other events.

It serves as a community that requires effective coordination and communication among its members. Traditionally, managing community information, maintaining member records, communicating important announcements, and organizing social events and campaigns may involve manual processes and scattered information. Such practices can make it difficult to maintain centralized records, ensure timely communication, manage event registrations efficiently, and provide members with convenient access to community-related information.

To address these challenges, the **KutumbSetu – Community Management System** has been proposed as a digital platform/app to simplify and organize community management activities. The application aims to provide features such as user registration and authentication, member profile management, community campaign and event information, and online participant or volunteer registration. By bringing these functionalities together in a centralized mobile application, KutumbSetu seeks to improve communication, streamline event management, maintain organized digital records, and strengthen coordination&relation among the members.

1.2 Problem Statement

Samast Darji Community, Babariyawad, requires an efficient system to manage community members, maintain records, communicate important announcements, and organize community events and campaigns. Managing these activities through traditional methods can make it difficult to maintain organized information, coordinate activities, and ensure timely communication among members.

At present, community-related information and activities may be managed through separate communication channels and manual processes. This fragmented approach can lead to:

- Difficulty in maintaining organized and updated member records.
- Delays in communicating important community announcements.
- Time-consuming management of events and campaigns.
- Challenges in handling participant and volunteer registrations.
- Possibility of duplicate records and errors in manual data management.

Moreover, the absence of a **centralized digital community management platform** can make it difficult to access information efficiently, manage registrations systematically, and provide members with convenient access to community activities.

To address these challenges, there is a need to develop **KutumbSetu-A Community Management System**, a mobile application designed to:

- Provide user registration and email OTP-based authentication.
- Maintain organized member profiles and information.
- Display community campaigns, events, and related details.
- Enable members to register as participants or volunteers.
- Support centralized storage and efficient management of community-related data.

Key Challenges Addressed:

- **Fragmented community information:** The lack of a centralized platform makes it difficult to maintain and access member records and community-related information efficiently.
- **Inefficient event and campaign management:** Handling registrations and coordinating participants through manual methods can increase administrative workload and create communication gaps.

By implementing KutumbSetu, the community can improve community coordination, simplify event and campaign registration, maintain organized digital records, and provide members with a more convenient way to access community-related information.

1.3 Proposed Solution

Proposed Solution

KutumbSetu –A Community Management System is a centralized mobile application designed to meet the community management needs of **the Community**. The project aims to simplify member registration, organize community information, and improve the management of community campaigns and activities through a single digital platform.

Key Features and Functionalities:

1. User Authentication and Profile Management: Provides email OTP-based authentication, member registration, unique Member ID generation, profile management, and administrator approval.

2. Community Directory and Family Tree: Enables members to search and filter community profiles by name, surname, native village, location, blood group, and profession, while visualizing multi-generational paternal and maternal family relationships.

3. Matrimonial Portal: Supports biodata creation, profile matching, connection requests, shortlisting, and matrimonial events to facilitate community matchmaking while protecting personal contact details.

4. Community Feed and Announcements: Displays community news, marriage announcements, birthdays, achievements, obituaries, and short videos, with administrative content moderation.

5. Campaign and Event Management: Allows administrators to publish campaigns and events, while members register as participants or volunteers, provide relevant details, and receive digital registration confirmations.

6. Blood Donor Registry: Maintains a searchable directory of voluntary blood donors based on blood group to help identify potential donors during emergencies.

7. Administrative Dashboard and Database Management: Provides authorized administrators with tools to manage members, announcements, campaigns, events, matrimonial requests, and community statistics, supported by centralized MongoDB storage.

8. Accessibility, Personalization: Offers English and Gujarati language support, dark and light themes, and a user-friendly interface.

This proposed system reduces dependence on scattered communication channels and manual record-keeping, improves the organization of member information, and simplifies campaign registration. By bringing essential community management functions together in one mobile application, KutumbSetu aims to strengthen coordination and make community activities more accessible to the members of the Community.

Chapter 2

2.1 System Requirements

The following are the minimum hardware and software requirements for installing and operating the **KutumbSetu** efficiently in a standard user environment.

Client-Side: Mobile Application

- **Device: Android smartphone or tablet.**
- **Operating System: Android 8.0 (Oreo) or higher.**
- **Internet: Wi-Fi or mobile data connection (2 Mbps or higher).**
- **Hardware Requirements:**
    - **RAM: 2 GB minimum; 4 GB recommended.**
    - **Processor: Quad-core processor or equivalent.**
    - **Storage: 150 MB of free internal storage.**
    - **Display: 720p resolution or higher.**
    - **Peripherals: Built-in camera for profile photo capture.**

Server-Side: Backend and Development Environment

- **Device: Laptop, desktop computer, or cloud server.**
- **Operating System: Windows 10/11 (64-bit) or Ubuntu Linux.**
- **Installed Software:**
    - **Node.js and npm for backend development.**
    - **MongoDB Community Server or MongoDB Atlas for database management.**
    - **Flutter SDK and Dart SDK for mobile application development.**
    - **Android SDK for building and testing the application.**
    - **Git for version control.**
    - **VS Code or Android Studio or Antigravity for development.**
    - **Postman for API testing.**

- **Hardware Requirements:**
    - **RAM: 8 GB minimum; 16 GB recommended.**
    - **Processor: Intel Core i3 (7th Generation or later), Intel Core i5, AMD Ryzen 5, or equivalent.**
    - **Storage: At least 100 GB available storage; SSD recommended.**
    - **Internet: Stable Wi-Fi or LAN connection for development, database access, and testing.**

2.2 Tools and Technologies

Backend: Node.js

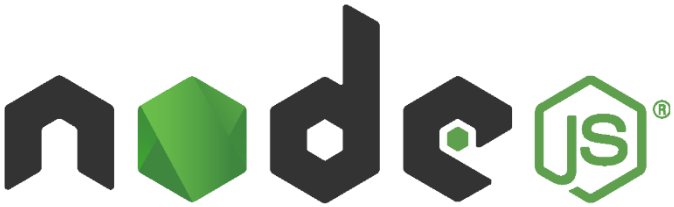


Figure 1. Node.js

Node.js is a powerful, open-source JavaScript runtime built on Chrome's V8 engine. It allows the development of fast and scalable server-side applications using asynchronous, event-driven architecture. This makes it ideal for handling multiple simultaneous data requests efficiently, a crucial requirement for a web-based data collection and reporting project like this. Node.js also has a vast ecosystem of libraries and frameworks that speed up development and integration with other services.

Frontend: Flutter



Figure 2. Flutter

Flutter **is an open-source UI software development kit (SDK)** created by Google. It deviates fundamentally from traditional cross-platform frameworks by operating like a game engine rather than a wrapper over native platform controls. Instead of translating its code into native iOS or Android UI components, Flutter bypasses the native platform's UI systems completely and renders every pixel directly onto an internal canvas.

Database: MongoDB Community Server Edition



Figure 3. MongoDB

MongoDB is a NoSQL, document-oriented database designed to handle large volumes of data with high availability and scalability. It stores data in flexible JSON-like documents, allowing easy handling of heterogeneous and evolving data formats — perfect for consolidating diverse data from multiple sources and directorates. Its robust querying capabilities and indexing features enable efficient data retrieval essential for report generation and analysis.

Tools

Android Studio



Figure 4. Android Studio

Android Studio is the official Integrated Development Environment (IDE) engineered by Google specifically to streamline the design, building, testing, and deployment of Android applications. Built upon JetBrains' powerful IntelliJ IDEA software, it modernizes the entire software development life cycle by consolidating a high-performance code editor—optimized for Kotlin, Java, and C++—with a robust, flexible Gradle-based build system. The environment accelerates developer workflows through integrated components like the Android SDK Manager for API lifecycle control, the high-speed Android Emulator for simulating diverse real-world hardware profiles, and an expansive suite of visual layout editors and intelligent performance profilers.

Postman



Figure 5. Postman

Postman is a popular collaboration platform used primarily for API development. It allows developers to design, test, and document APIs efficiently. With Postman, users can send various types of HTTP requests (GET, POST, PUT, DELETE, etc.), inspect responses, and automate testing workflows. It supports features like environment variables, scripting, and collections to organize API calls. Postman simplifies working with APIs by providing a user-friendly interface, making it easier to debug, share, and collaborate on API projects within teams.

Antigravity

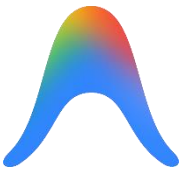


Figure 5. Antigravity

Antigravity is an AI-powered development environment designed to assist software development through AI agents. It can help developers understand code, generate and modify program files, identify errors, and automate certain development tasks based on instructions.

Git&Github





Figure 6.Git&Github

Git is a free, open-source distributed version control system (VCS) created by Linus Torvalds in 2005. It runs locally on your machine to track history, changes, and snapshots in your code. GitHub is a cloud-based web hosting platform for Git repositories (owned by Microsoft) that adds a graphical interface and tools for online storage, teamwork, and code review

Figma



Figure 7.Figma

Figma is a cloud-based interface design and prototyping tool used to create wireframes, user interface layouts, interactive prototypes, and design systems. It enables designers and developers to collaborate, share designs, and maintain consistent visual elements throughout an application.

2.3 Features

- **Centralized Mobile Application:** Provides a unified platform for community members to access their profiles, community information, family tree, matrimonial services, and campaigns.
- **User Authentication and Registration:** Supports email OTP-based authentication, member registration, unique Member ID generation, and administrator approval.
- **Community Directory**: Enables members to search and filter profiles based on name, surname, native village, location, blood group, and profession.
- **Interactive Family Tree:** Displays multi-generational paternal and maternal family relationships in a visual format.
- **Matrimonial Portal:** Provides biodata creation, profile matching, connection requests, shortlisting, and matrimonial event information.
- **Community Feed and Announcements:** Displays community news, marriage announcements, birthdays, achievements, obituaries, and short videos, with content moderation.
- **Campaign and Event Management:** Allows administrators to publish campaigns and members to register as participants or volunteers, with registration details and confirmations.
- **Blood Donor Registry:** Helps members search for voluntary blood donors based on blood group for emergency requirements.
- **Administrative Dashboard**: Enables authorized administrators to manage member approvals, announcements, campaigns, events, profiles, matrimonial requests, and community statistics.
- **Language and Theme Customization:** Supports English and Gujarati languages, along with dark and light modes for a personalized user experience.
- **Centralized Database Management**: Uses **MongoDB** to store and manage member profiles, campaign registrations, and other community information.

2.4 Gantt Chart

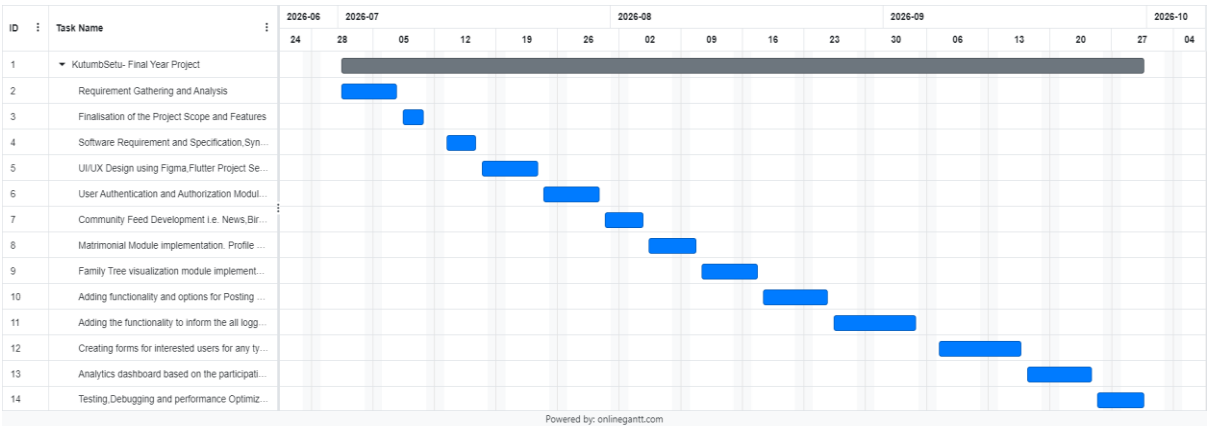


Figure 8. Gantt Chart

Chapter 3

3.1 SYSTEM DESIGN

3.1.1 Flowchart

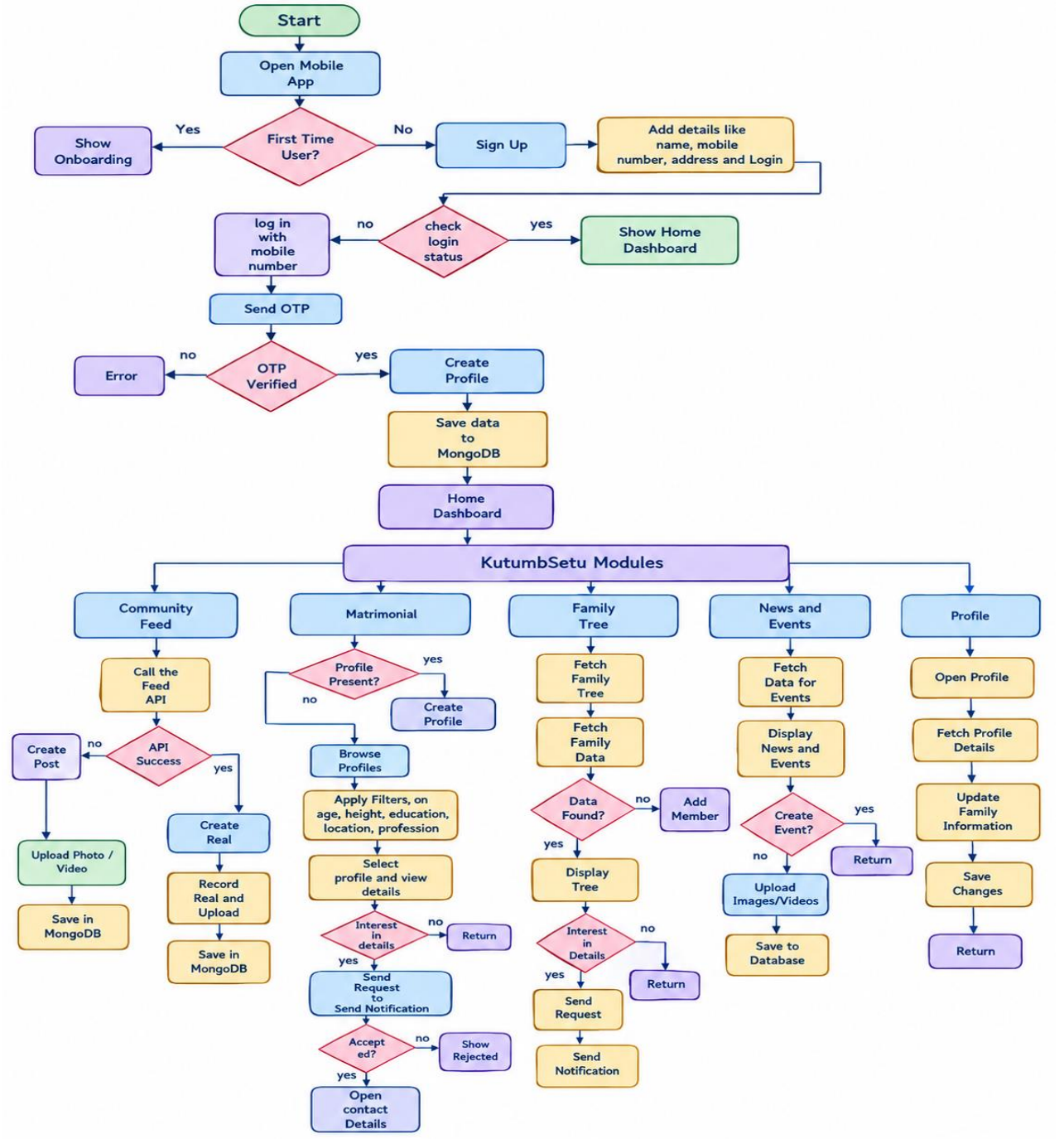


Figure 9. Flow Chart

3.1.2 DFD Level 0:

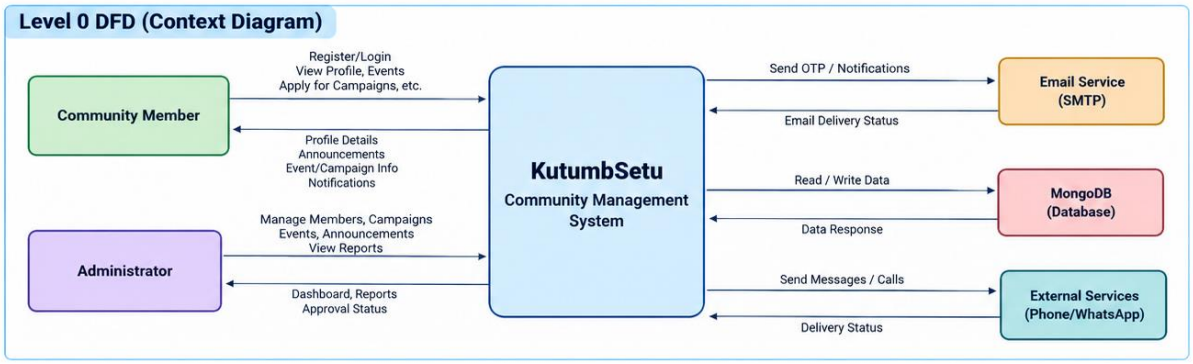


Figure 10. Level 0

3.1.3 DFD Level 1:

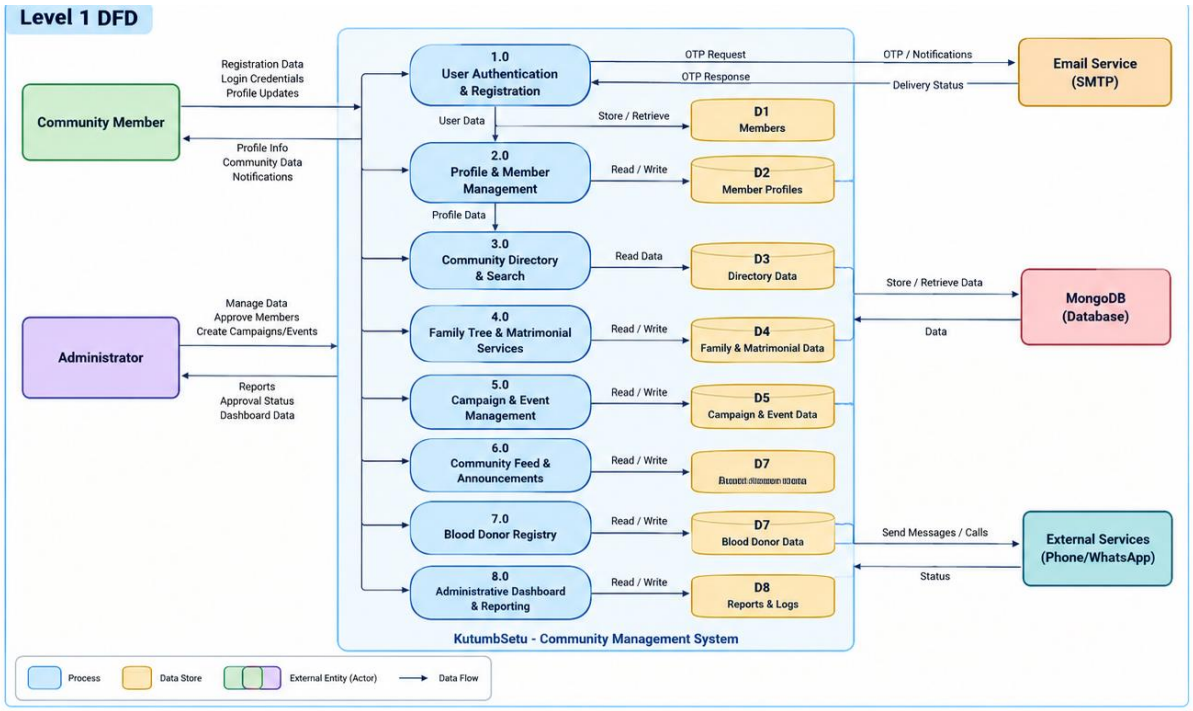


Figure 10. Level 1

3.1.4 Use Case Diagram

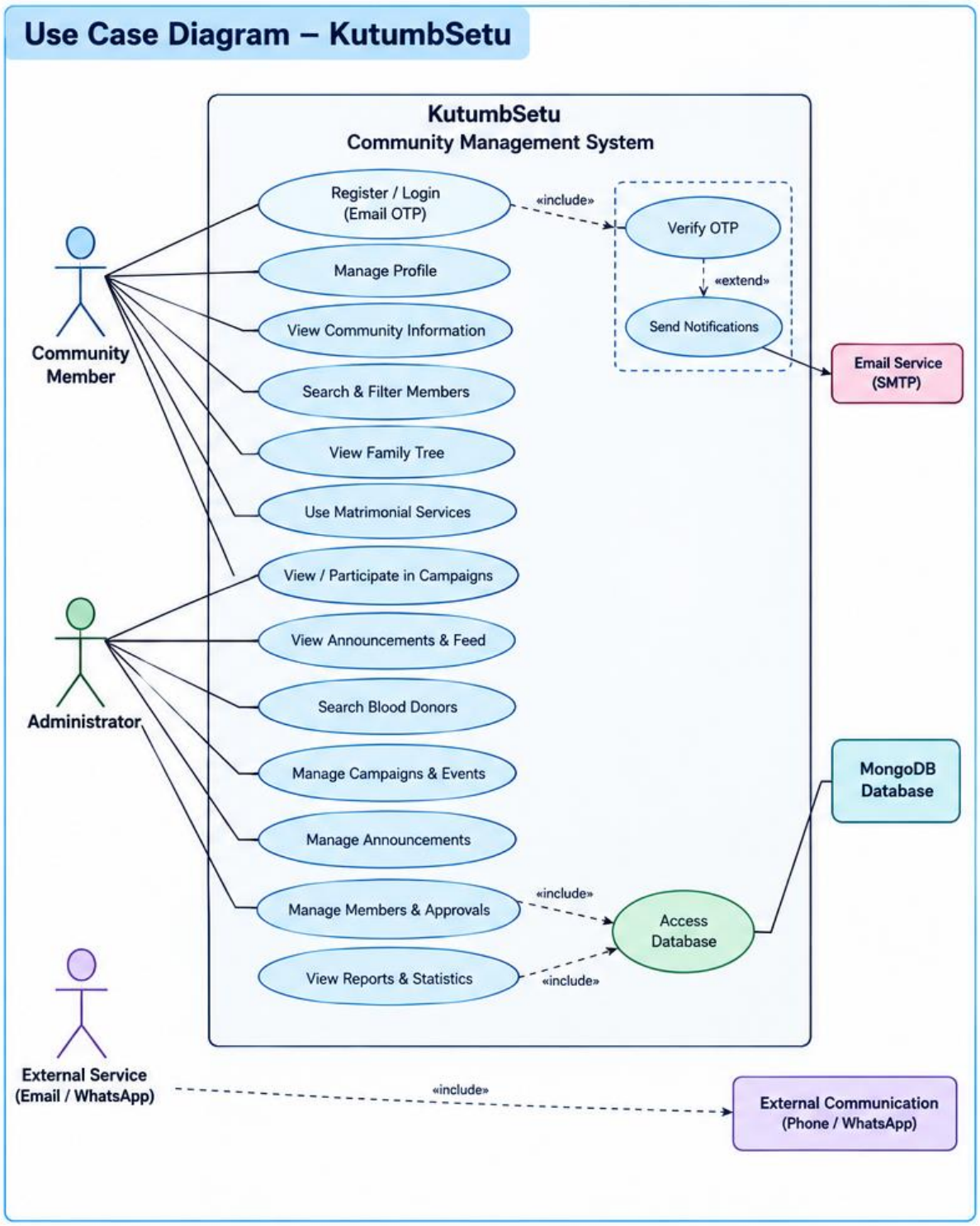


Figure 11. Use Case Diagram

3.1.5 **UML Activity Diagram**

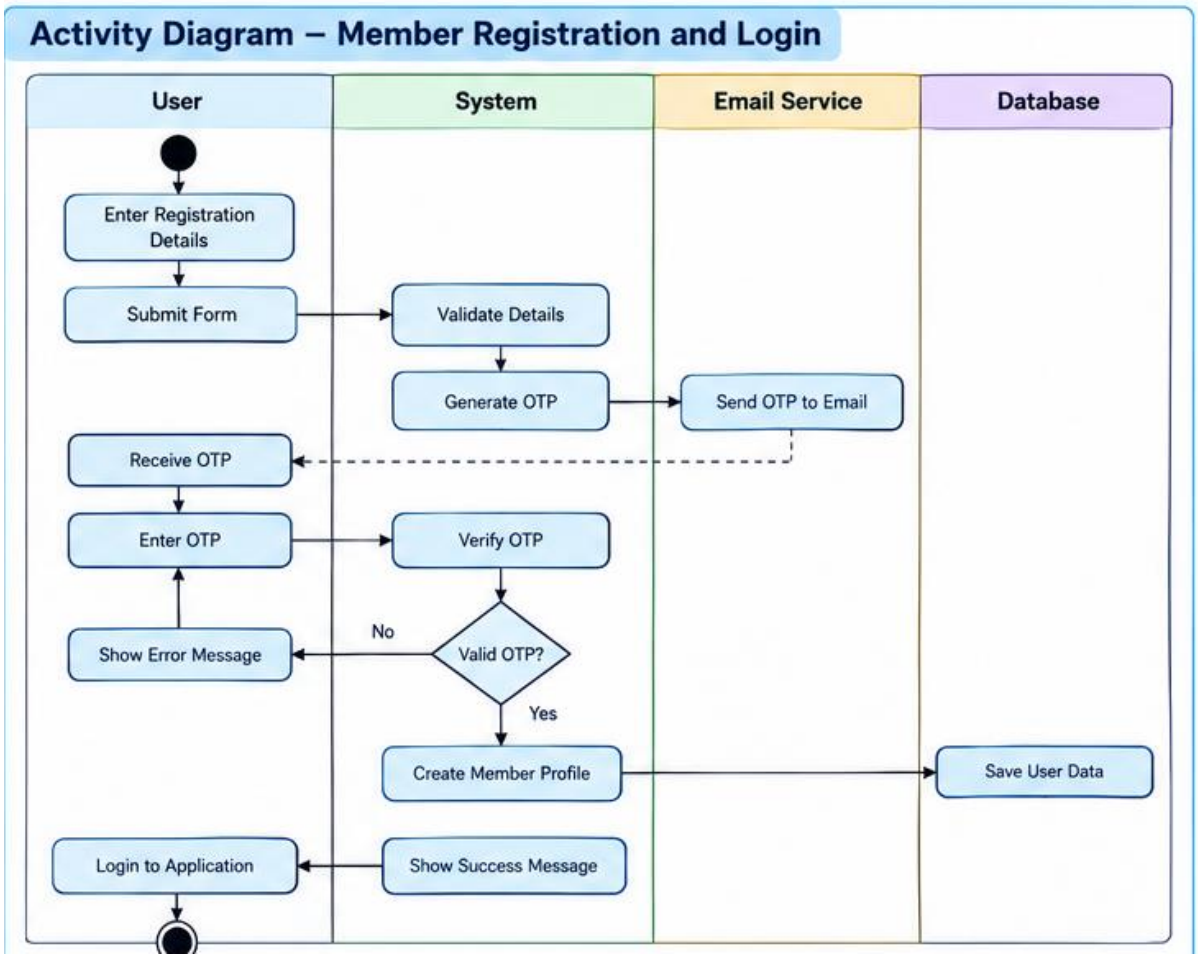


Figure 12. Activity Diagram

3.1.6 Component Diagram

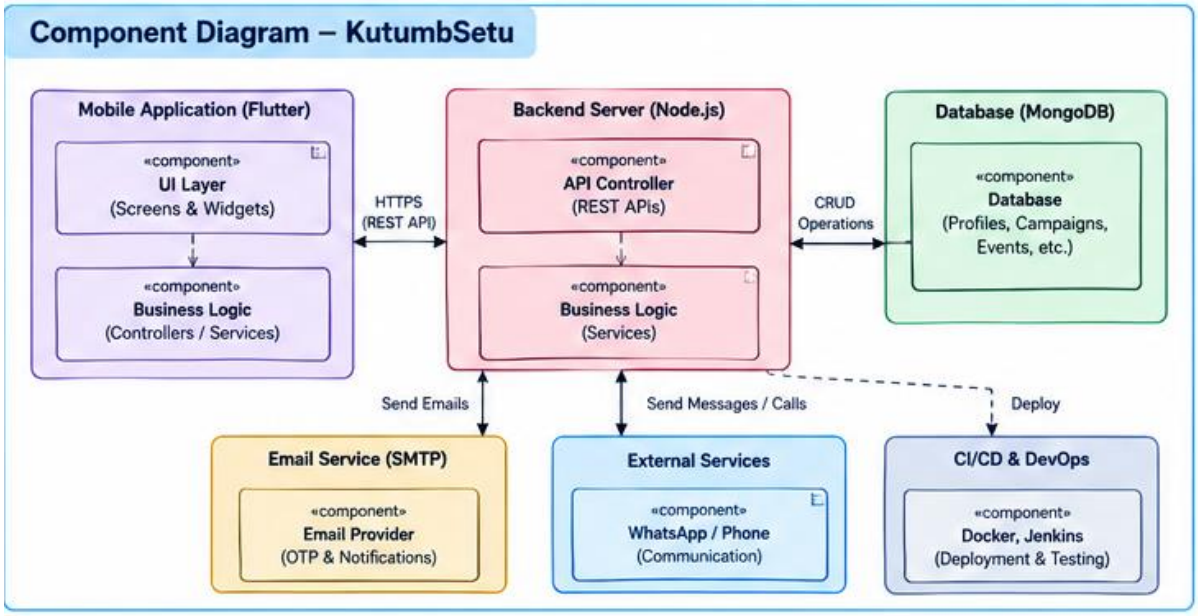


Figure 13. Component Diagram

3.1.7 Deployment Diagram

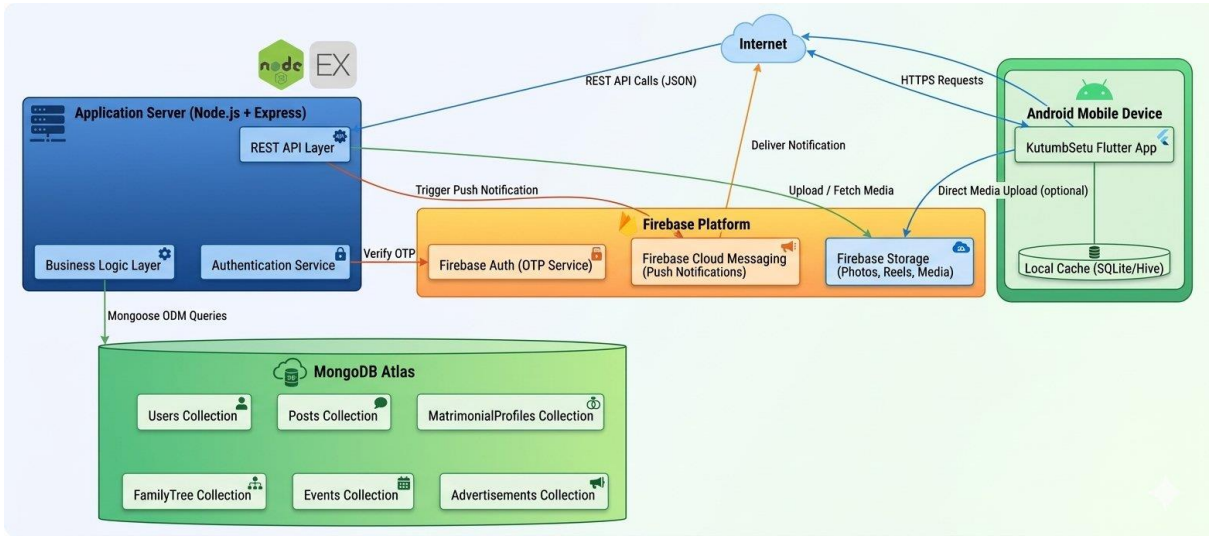


Figure 14. Deployment Diagram

Chapter 4

4.1 Literature Review

1. Stakeholder Consultation and Community Requirement Analysis:

The initial phase of our study involved a comprehensive assessment of traditional community management methodologies, existing paper-based member directories **(*Vasti Patrak*), and manual matrimonial registries.** To gain deeper insights into the organizational workflows, ground realities, and digital requirements of the community, we conducted multiple strategic meetings **with the Head of the Community, **Mr. Ramesh Parmar**, along with senior executive committee members including **Mr. Jagdish Parmar**, **Mr. Rakesh Parmar**, and other community elders.** These collaborative sessions allowed us to thoroughly analyze the operational bottlenecks of manual record-keeping, such as fragmented demographic records across dispersed cities, delays in broadcasting urgent announcements (e.g., emergency blood donation appeals), and privacy vulnerabilities associated with circulating matrimonial biodatas on public social media groups**. Based on these discussions with Mr. Ramesh Parmar and the executive team, we systematically formulated the functional specifications, finalized the project scope, and defined the architectural roadmap for **KutumbSetu**, ensuring that the digital platform adheres directly to community values, institutional governance, and privacy expectations.**

2. Study of Hierarchical Genealogical Modeling and Family Tree Algorithms:

To design an interactive multi-generational genealogical visualization system,we conducted an in-depth study of hierarchical tree structures, parent-child relational mappings, and recursive graph traversal models. Traditional genealogical representations are predominantly static, paper-based, or **limited to single-lineage paternal tracking.** Through our literature survey of graph rendering methodologies, we examined the integration of custom canvas rendering engines and viewport controllers (utilizing Flutter's `TransformationController` and vector coordinate positioning). This study enabled us to implement a responsive, infinite-canvas family tree capable **of seamless panning and multi-level pinch-to-zoom.** Furthermore, our research into relational data modeling led to the development of a dual-lineage structure that dynamically visualizes both Paternal and Maternal lineages while employing fuzzy name equivalence algorithms to prevent duplicate ancestral node creation.

3. Investigation of Privacy-Preserving Matrimonial Matchmaking Architectures:

During our review of contemporary matrimonial portals, we analyzed existing commercial solutions such as **Shaadi.com, BharatMatrimony**, and localized community matchmaking booklets. While commercial platforms offer extensive reach, they often lack community-centric verification and present serious privacy challenges, as members' private contact numbers and personal photographs are frequently exposed to unsolicited inquiries. Our review focused on **developing a privacy-first, two-tier consent gatekeeper**. Under this architecture, candidate biodatas (covering education, career, gotra, and lifestyle attributes) are openly browsable within the verified community, while direct contact details (phone numbers and WhatsApp channels) remain strictly masked until a formal match request is reciprocated and

accepted by both parties. This research significantly enhanced the **safety, dignity, and acceptance** of the matrimonial module among community families.

4. Analysis of Secure Authentication, Identity Verification, and Role Governance:

For building a trusted community ecosystem, we studied various digital onboarding protocols, tokenization models, and authentication workflows. Given that community applications are prone to unauthorized access, impersonation, and spam registrations, our literature review concentrated on multi-factor verification techniques and state management architectures. We examined the implementation of **automated 6-digit One-Time Password (OTP) validation delivered via Simple Mail Transfer Protocol (SMTP) using Nodemailer, ensuring that every registered user holds an authentic digital footprint**. Additionally, we analyzed **administrative gatekeeping mechanisms** and **Role-Based Access** Control (RBAC). By evaluating these security patterns, we established a quarantine workflow wherein newly registered profiles remain in a "Pending Verification" queue until physically or digitally authenticated by the administrative committee, maintaining absolute integrity within the portal.

5. Comparative Study of Existing Community Portals and Social Platforms:

To benchmark industry standards and understand the limitations of prevailing digital alternatives, we performed a comparative study of generic platforms like the **Kutumb App, WhatsApp Community Groups, and localized directory software**. While platforms like **WhatsApp** facilitate instant communication, they completely lack structured demographic search, blood-donor filtering, historical archiving, and genealogical tracing. Generic community applications often impose intrusive commercial advertisements and lack tailored cultural integration. By examining these drawbacks, we gained valuable architectural insights into creating an integrated, ad-free, and bilingual ecosystem (supporting both English and Gujarati). This study guided our functional decisions to unify a **searchable member directory, emergency blood registry, moderated community feed** and administrative command center into a single, cohesive application framework.

4.2 Source Code

lib/main.dart

```
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:google_fonts/google_fonts.dart';
import 'providers/theme_provider.dart';
import 'routes/app_router.dart';
import 'theme.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  GoogleFonts.config.allowRuntimeFetching = false;
  try {
    await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  } catch (e) {
    print('Firebase initialization skipped or failed: $e');
  }
  runApp(
    const ProviderScope(
      child: KutumbSetuApp(),
    ),
  );
}

class KutumbSetuApp extends ConsumerWidget {
  const KutumbSetuApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final themeMode = ref.watch(themeModeProvider);

    return MaterialApp.router(
      title: 'KutumbSetu',
      debugShowCheckedModeBanner: false,
      theme: KutumbSetuTheme.lightTheme,
      darkTheme: KutumbSetuTheme.darkTheme,
      themeMode: themeMode,
      routerConfig: appRouter,
    );
  }
}
```

lib/api_config.dart

```
import 'dart:io';
import 'package:flutter/foundation.dart';

class ApiConfig {
  /// Toggle this to [true] only if testing with USB cable ADB reverse.
  /// When using the deployed cloud backend, keep this [false].
  static const bool useAdbReverseForPhysicalAndroid = true;

  /// Deployed cloud backend URL (Render.com)
  static const String publicTunnelUrl = '';

  /// Alternatively, enter your computer's local Wi-Fi IP address here (e.g., '192.168.1.15')
  static const String localWifiIp = '';

  /// Base API URL pointing to the local Node.js Express server.
  static String get baseUrl {
    if (publicTunnelUrl.isNotEmpty) {
      return '$publicTunnelUrl/api';
    }
    if (!kIsWeb && Platform.isAndroid && useAdbReverseForPhysicalAndroid) {
      return 'http://127.0.0.1:5000/api';
    }
    if (localWifiIp.isNotEmpty) {
      return 'http://$localWifiIp:5000/api';
    }
    if (!kIsWeb && Platform.isAndroid) {
      return 'http://10.0.2.2:5000/api';
    }
    return 'http://localhost:5000/api';
  }
}
```

lib/routes/app_router.dart

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import '../models/campaign_model.dart';
import '../models/member_model.dart';
import '../screens/login_screen.dart';
import '../screens/register_screen.dart';
import '../screens/home_screen.dart';
import '../screens/member_directory_screen.dart';
import '../screens/member_profile_screen.dart';
import '../screens/matrimonial/matrimonial_hub_screen.dart';
```

```
import '../screens/matrimonial/matrimonial_profile_list_screen.dart';
import '../screens/matrimonial/matrimonial_profile_detail_screen.dart';
import '../screens/matrimonial/matrimonial_biodata_screen.dart';
import '../screens/matrimonial/matrimonial_requests_screen.dart';
import '../screens/matrimonial/marriage_events_screen.dart';
import '../screens/matrimonial/success_stories_screen.dart';
import '../screens/matrimonial/matrimonial_admin_screen.dart';
import '../screens/matrimonial/premium_benefits_screen.dart';
import '../screens/profile_completion_screen.dart';
import '../screens/admin_registrations_screen.dart';
import '../screens/campaign_create_screen.dart';
import '../screens/campaign_detail_screen.dart';
import '../screens/campaign_listing_screen.dart';
import '../screens/campaign_registration_screen.dart';
import '../screens/my_registrations_screen.dart';
import '../screens/registration_success_screen.dart';
import '../screens/post_detail_screen.dart';
import '../screens/reel_detail_screen.dart';
import '../screens/admin_dashboard_screen.dart';

final appRouter = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/admin',
      builder: (context, state) => const AdminDashboardScreen(),
    ),
    GoRoute(
      path: '/post/:id',
      builder: (context, state) {
        final id = state.pathParameters['id'] ?? '';
        return PostDetailScreen(postId: id);
      },
    ),
    GoRoute(
      path: '/reel/:id',
      builder: (context, state) {
        final id = state.pathParameters['id'] ?? '';
        return ReelDetailScreen(reelId: id);
      },
    ),
    GoRoute(
      path: '/',
      builder: (context, state) => const LoginScreen(),
    ),
    GoRoute(
      path: '/register',
      builder: (context, state) => const RegisterScreen(),
```

```
),
GoRoute(
  path: '/profile/complete',
  builder: (context, state) => const ProfileCompletionScreen(),
),
GoRoute(
  path: '/home',
  builder: (context, state) => const HomeScreen(),
),
GoRoute(
  path: '/directory',
  builder: (context, state) => const MemberDirectoryScreen(),
),
GoRoute(
  path: '/member/:id',
  builder: (context, state) {
    final member = state.extra as Member?;
    final id = state.pathParameters['id'] ?? '';
    return MemberProfileScreen(memberId: id, member: member);
  },
),
// Matrimonial Routes
GoRoute(
  path: '/matrimonial',
  builder: (context, state) => const MatrimonialHubScreen(),
),
GoRoute(
  path: '/matrimonial/profiles',
  builder: (context, state) {
    final shortlisted = state.uri.queryParameters['shortlisted'] == 'true';
    final recommendations = state.uri.queryParameters['recommendations'] == 'true';
    return MatrimonialProfileListScreen(
      showShortlistedOnly: shortlisted,
      showRecommendationsOnly: recommendations,
    );
  },
),
GoRoute(
  path: '/matrimonial/profile/:id',
  builder: (context, state) {
    final id = state.pathParameters['id'] ?? '';
    return MatrimonialProfileDetailScreen(userId: id);
  },
),
GoRoute(
  path: '/matrimonial/biodata',
  builder: (context, state) => const MatrimonialBiodataScreen(),
),
```

```
GoRoute(
  path: '/matrimonial/requests',
  builder: (context, state) => const MatrimonialRequestsScreen(),
),
GoRoute(
  path: '/matrimonial/events',
  builder: (context, state) => const MarriageEventsScreen(),
),
GoRoute(
  path: '/matrimonial/stories',
  builder: (context, state) => const SuccessStoriesScreen(),
),
GoRoute(
  path: '/matrimonial/admin',
  builder: (context, state) => const MatrimonialAdminScreen(),
),
GoRoute(
  path: '/matrimonial/premium-benefits',
  builder: (context, state) => const PremiumBenefitsScreen(),
),
// Campaign Routes
GoRoute(
  path: '/campaigns',
  builder: (context, state) => const CampaignListingScreen(),
),
GoRoute(
  path: '/campaigns/create',
  builder: (context, state) => const CampaignCreateScreen(),
),
GoRoute(
  path: '/campaigns/:id',
  builder: (context, state) {
    final campaign = state.extra as Campaign?;
    final id = state.pathParameters['id'] ?? '';
    return CampaignDetailScreen(campaignId: id, initialCampaign: campaign);
  },
),
GoRoute(
  path: '/campaigns/:id/register',
  builder: (context, state) {
    final campaign = state.extra as Campaign?;
    final id = state.pathParameters['id'] ?? '';
    return CampaignRegistrationScreen(campaignId: id, campaign: campaign);
  },
),
GoRoute(
  path: '/campaigns/:id/success',
  builder: (context, state) {
```

```
        final regData = state.extra as Map<String, dynamic>? ?? {};
        return RegistrationSuccessScreen(registrationData: regData);
      },
    ),
    GoRoute(
      path: '/campaigns/:id/registrations',
      builder: (context, state) {
        final id = state.pathParameters['id'] ?? '';
        return AdminRegistrationsScreen(campaignId: id);
      },
    ),
    GoRoute(
      path: '/my-registrations',
      builder: (context, state) => const MyRegistrationsScreen(),
    ),
  ],
  errorBuilder: (context, state) => Scaffold(
    body: Center(
      child: Text('Page not found: ${state.error}'),
    ),
  ),
);
```

lib/providers/auth_provider.dart

```
import 'dart:convert';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:http/http.dart' as http;
import '../api_config.dart';
import '../models/user_model.dart';

// StateProvider used by the Directory, Matrimonial, and Family Tree features
final currentUserProvider = StateProvider<UserModel?>((ref) => null);

class AuthService {
  static Future<UserModel?> fetchUserProfile(String phoneNumber) async {
    try {
      final sanitizedPhone = phoneNumber.replaceAll(' ', '').trim();
      final response = await http.get(
        Uri.parse('${ApiConfig.baseUrl}/users/profile/$sanitizedPhone'),
        headers: {'Content-Type': 'application/json'},
      );
      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        if (data['success'] == true && data['user'] != null) {
          return UserModel.fromJson(data['user']);
        }
```

```
      }
    } catch (e) {
      print('Error fetching user profile: $e');
    }
    return null;
  }
}

class AuthState {
  final UserModel? user;
  final bool isLoading;
  final String? error;

  const AuthState({
    this.user,
    this.isLoading = false,
    this.error,
  });

  bool get isAuthenticated => user != null;
  bool get isAdmin => user?.isAdmin ?? false;

  AuthState copyWith({
    UserModel? user,
    bool? isLoading,
    String? error,
    bool clearUser = false,
  }) {
    return AuthState(
      user: clearUser ? null : (user ?? this.user),
      isLoading: isLoading ?? this.isLoading,
      error: error,
    );
  }
}

class AuthNotifier extends StateNotifier<AuthState> {
  final Ref ref;

  AuthNotifier(this.ref) : super(const AuthState()) {
    final current = ref.read(currentUserProvider);
    if (current != null) {
      state = state.copyWith(user: current);
    }
  }

  Future<void> fetchUserByPhone(String phone) async {
    state = state.copyWith(isLoading: true, error: null);
```

```
  try {
    final sanitizedPhone = phone.replaceAll(' ', '').trim();
    final response = await http
        .get(Uri.parse('${ApiConfig.baseUrl}/users/by-phone/$sanitizedPhone'))
        .timeout(const Duration(seconds: 5));

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      if (data['success'] == true && data['user'] != null) {
        final user = UserModel.fromJson(data['user'] as Map<String, dynamic>);
        state = state.copyWith(user: user, isLoading: false);
        ref.read(currentUserProvider.notifier).state = user;
        return;
      }
    }
    state = state.copyWith(isLoading: false);
  } catch (e) {
    state = state.copyWith(isLoading: false);
  }
}

void setUser(UserModel user) {
  if (state.user != user) {
    state = state.copyWith(user: user);
  }
}

void toggleAdminRole() async {
  if (state.user == null) return;
  final currentRole = state.user!.role;
  final newRole = (currentRole == 'admin' || currentRole == 'organizer') ? 'user' : 'admin';
  final updatedUser = state.user!.copyWith(role: newRole);

  state = state.copyWith(user: updatedUser);
  ref.read(currentUserProvider.notifier).state = updatedUser;

  try {
    await http.patch(
      Uri.parse('${ApiConfig.baseUrl}/users/${updatedUser.id}/role'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'role': newRole}),
    );
  } catch (_) {}
}

void logout() {
  state = state.copyWith(clearUser: true);
  ref.read(currentUserProvider.notifier).state = null;
```

```
  }
}

// StateNotifierProvider used by the Campaign feature
final authProvider = StateNotifierProvider<AuthNotifier, AuthState>((ref) {
  final notifier = AuthNotifier(ref);

  // Sync authProvider state when currentUserProvider is mutated directly (e.g. at login screen)
  ref.listen<UserModel?>(currentUserProvider, (previous, next) {
    if (next != null) {
      notifier.setUser(next);
    } else {
      notifier.logout();
    }
  });

  return notifier;
});
```

lib/providers/member_providers.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../api_config.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../models/member_model.dart';
import '../repository/member_repository.dart';

enum SortOption {
  nameAZ,
  nameZA,
  profession,
  city,
  village,
  newestJoined,
  oldestJoined,
  verifiedFirst,
}

extension SortOptionExtension on SortOption {
  String get label {
    switch (this) {
      case SortOption.nameAZ:
        return 'Alphabetical (A - Z)';
      case SortOption.nameZA:
        return 'Alphabetical (Z - A)';
      case SortOption.profession:
```

```
        return 'Profession';
      case SortOption.city:
        return 'City';
      case SortOption.village:
        return 'Village';
      case SortOption.newestJoined:
        return 'Newest Joined';
      case SortOption.oldestJoined:
        return 'Oldest Joined';
      case SortOption.verifiedFirst:
        return 'Verified First';
    }
  }
}

class MemberFilterState {
  final Set<String> selectedVillages;
  final Set<String> selectedCities;
  final Set<String> selectedProfessions;
  final Set<String> selectedEducations;
  final Set<String> selectedBusinessCategories;
  final Set<String> selectedBloodGroups;
  final Set<String> selectedGenders;
  final bool? isVerifiedOnly;
  final bool? isActiveOnly;

  const MemberFilterState({
    this.selectedVillages = const {},
    this.selectedCities = const {},
    this.selectedProfessions = const {},
    this.selectedEducations = const {},
    this.selectedBusinessCategories = const {},
    this.selectedBloodGroups = const {},
    this.selectedGenders = const {},
    this.isVerifiedOnly,
    this.isActiveOnly,
  });

  bool get isEmpty =>
      selectedVillages.isEmpty &&
      selectedCities.isEmpty &&
      selectedProfessions.isEmpty &&
      selectedEducations.isEmpty &&
      selectedBusinessCategories.isEmpty &&
      selectedBloodGroups.isEmpty &&
      selectedGenders.isEmpty &&
      isVerifiedOnly == null &&
      isActiveOnly == null;
```

```
  int get activeFilterCount {
    int count = 0;
    count += selectedVillages.length;
    count += selectedCities.length;
    count += selectedProfessions.length;
    count += selectedEducations.length;
    count += selectedBusinessCategories.length;
    count += selectedBloodGroups.length;
    count += selectedGenders.length;
    if (isVerifiedOnly != null) count += 1;
    if (isActiveOnly != null) count += 1;
    return count;
  }

  MemberFilterState copyWith({
    Set<String>? selectedVillages,
    Set<String>? selectedCities,
    Set<String>? selectedProfessions,
    Set<String>? selectedEducations,
    Set<String>? selectedBusinessCategories,
    Set<String>? selectedBloodGroups,
    Set<String>? selectedGenders,
    bool? isVerifiedOnly,
    bool? isActiveOnly,
    bool clearVerified = false,
    bool clearActive = false,
  }) {
    return MemberFilterState(
      selectedVillages: selectedVillages ?? this.selectedVillages,
      selectedCities: selectedCities ?? this.selectedCities,
      selectedProfessions: selectedProfessions ?? this.selectedProfessions,
      selectedEducations: selectedEducations ?? this.selectedEducations,
      selectedBusinessCategories:
          selectedBusinessCategories ?? this.selectedBusinessCategories,
      selectedBloodGroups: selectedBloodGroups ?? this.selectedBloodGroups,
      selectedGenders: selectedGenders ?? this.selectedGenders,
      isVerifiedOnly:
          clearVerified ? null : (isVerifiedOnly ?? this.isVerifiedOnly),
      isActiveOnly: clearActive ? null : (isActiveOnly ?? this.isActiveOnly),
    );
  }
}

class MemberFilterNotifier extends StateNotifier<MemberFilterState> {
  MemberFilterNotifier() : super(const MemberFilterState());

  void toggleVillage(String village) {
```

```
  final updated = Set<String>.from(state.selectedVillages);
  if (updated.contains(village)) {
    updated.remove(village);
  } else {
    updated.add(village);
  }
  state = state.copyWith(selectedVillages: updated);
}

void toggleCity(String city) {
  final updated = Set<String>.from(state.selectedCities);
  if (updated.contains(city)) {
    updated.remove(city);
  } else {
    updated.add(city);
  }
  state = state.copyWith(selectedCities: updated);
}

void toggleProfession(String profession) {
  final updated = Set<String>.from(state.selectedProfessions);
  if (updated.contains(profession)) {
    updated.remove(profession);
  } else {
    updated.add(profession);
  }
  state = state.copyWith(selectedProfessions: updated);
}

void toggleEducation(String education) {
  final updated = Set<String>.from(state.selectedEducations);
  if (updated.contains(education)) {
    updated.remove(education);
  } else {
    updated.add(education);
  }
  state = state.copyWith(selectedEducations: updated);
}

void toggleBusinessCategory(String category) {
  final updated = Set<String>.from(state.selectedBusinessCategories);
  if (updated.contains(category)) {
    updated.remove(category);
  } else {
    updated.add(category);
  }
  state = state.copyWith(selectedBusinessCategories: updated);
}
```

```
  void toggleBloodGroup(String bg) {
    final updated = Set<String>.from(state.selectedBloodGroups);
    if (updated.contains(bg)) {
      updated.remove(bg);
    } else {
      updated.add(bg);
    }
    state = state.copyWith(selectedBloodGroups: updated);
  }

  void toggleGender(String gender) {
    final updated = Set<String>.from(state.selectedGenders);
    if (updated.contains(gender)) {
      updated.remove(gender);
    } else {
      updated.add(gender);
    }
    state = state.copyWith(selectedGenders: updated);
  }

  void toggleVerifiedOnly() {
    if (state.isVerifiedOnly == true) {
      state = state.copyWith(clearVerified: true);
    } else {
      state = state.copyWith(isVerifiedOnly: true);
    }
  }

  void toggleActiveOnly() {
    if (state.isActiveOnly == true) {
      state = state.copyWith(clearActive: true);
    } else {
      state = state.copyWith(isActiveOnly: true);
    }
  }

  void clearAll() {
    state = const MemberFilterState();
  }
}

class FavoriteNotifier extends StateNotifier<Set<String>> {
  FavoriteNotifier() : super({});

  void toggleFavorite(String memberId) {
    final updated = Set<String>.from(state);
    if (updated.contains(memberId)) {
```

```
      updated.remove(memberId);
    } else {
      updated.add(memberId);
    }
    state = updated;
  }
}

// Providers definition
final memberRepositoryProvider = Provider<MemberRepository>((ref) {
  return LocalMemberRepository();
});

final rawMemberListProvider = FutureProvider<List<Member>>((ref) async {
  final repo = ref.watch(memberRepositoryProvider);
  return repo.getMembers();
});

final searchQueryProvider = StateProvider<String>((ref) => '');

final sortOptionProvider =
    StateNotifierProvider<SortNotifier, SortOption>((ref) {
  return SortNotifier();
});

class SortNotifier extends StateNotifier<SortOption> {
  SortNotifier() : super(SortOption.nameAZ);

  void setSortOption(SortOption option) {
    state = option;
  }
}

final activeFiltersProvider =
    StateNotifierProvider<MemberFilterNotifier, MemberFilterState>((ref) {
  return MemberFilterNotifier();
});

final favoriteMemberIdsProvider =
    StateNotifierProvider<FavoriteNotifier, Set<String>>((ref) {
  return FavoriteNotifier();
});

final filteredMembersProvider = Provider<AsyncValue<List<Member>>>((ref) {
  final rawAsync = ref.watch(rawMemberListProvider);
  final query = ref.watch(searchQueryProvider).trim().toLowerCase();
  final filters = ref.watch(activeFiltersProvider);
  final sortOption = ref.watch(sortOptionProvider);
```

```
return rawAsync.whenData((members) {
  List<Member> result = List.from(members);

  // 1. Search Filter
  if (query.isNotEmpty) {
    result = result.where((m) {
      return m.fullName.toLowerCase().contains(query) ||
          m.email.toLowerCase().contains(query) ||
          m.village.toLowerCase().contains(query) ||
          m.city.toLowerCase().contains(query) ||
          m.profession.toLowerCase().contains(query) ||
          m.education.toLowerCase().contains(query) ||
          m.company.toLowerCase().contains(query) ||
          m.mobileNumber.replaceAll(' ', '').contains(query);
    }).toList();
  }

  // 2. Attribute Filters
  if (filters.selectedVillages.isNotEmpty) {
    result = result
        .where((m) => filters.selectedVillages.contains(m.village))
        .toList();
  }

  if (filters.selectedCities.isNotEmpty) {
    result =
        result.where((m) => filters.selectedCities.contains(m.city)).toList();
  }

  if (filters.selectedProfessions.isNotEmpty) {
    result = result
        .where((m) => filters.selectedProfessions.contains(m.profession))
        .toList();
  }

  if (filters.selectedEducations.isNotEmpty) {
    result = result
        .where((m) => filters.selectedEducations.contains(m.education))
        .toList();
  }

  if (filters.selectedBusinessCategories.isNotEmpty) {
    result = result
        .where((m) =>
            filters.selectedBusinessCategories.contains(m.businessCategory))
        .toList();
  }
```

```
if (filters.selectedBloodGroups.isNotEmpty) {
  result = result
      .where((m) => filters.selectedBloodGroups.contains(m.bloodGroup))
      .toList();
}

if (filters.selectedGenders.isNotEmpty) {
  result = result
      .where((m) => filters.selectedGenders.contains(m.gender))
      .toList();
}

if (filters.isVerifiedOnly == true) {
  result = result.where((m) => m.isVerified).toList();
}

if (filters.isActiveOnly == true) {
  result = result.where((m) => m.isActive).toList();
}

// 3. Sorting
switch (sortOption) {
  case SortOption.nameAZ:
    result.sort((a, b) =>
        a.fullName.toLowerCase().compareTo(b.fullName.toLowerCase()));
    break;
  case SortOption.nameZA:
    result.sort((a, b) =>
        b.fullName.toLowerCase().compareTo(a.fullName.toLowerCase()));
    break;
  case SortOption.profession:
    result.sort((a, b) =>
        a.profession.toLowerCase().compareTo(b.profession.toLowerCase()));
    break;
  case SortOption.city:
    result.sort(
        (a, b) => a.city.toLowerCase().compareTo(b.city.toLowerCase()));
    break;
  case SortOption.village:
    result.sort((a, b) =>
        a.village.toLowerCase().compareTo(b.village.toLowerCase()));
    break;
  case SortOption.newestJoined:
    result.sort((a, b) => b.joinedDate.compareTo(a.joinedDate));
    break;
  case SortOption.oldestJoined:
    result.sort((a, b) => a.joinedDate.compareTo(b.joinedDate));
```

```
        break;
      case SortOption.verifiedFirst:
        result.sort((a, b) {
          if (a.isVerified && !b.isVerified) return -1;
          if (!a.isVerified && b.isVerified) return 1;
          return a.fullName.toLowerCase().compareTo(b.fullName.toLowerCase());
        });
        break;
    }

    return result;
  });
});

final groupedMembersProvider =
    Provider<AsyncValue<Map<String, List<Member>>>>((ref) {
  final filteredAsync = ref.watch(filteredMembersProvider);

  return filteredAsync.whenData((members) {
    final Map<String, List<Member>> grouped = {};
    for (final member in members) {
      final letter = member.firstLetter;
      grouped.putIfAbsent(letter, () => []).add(member);
    }
    return grouped;
  });
});

// Directory Connected / Followed Members State Management
class DirectoryConnectionState {
  final Set<String> connectedMemberIds;
  final Set<String> pendingRequestMemberIds;
  final bool isLoading;

  const DirectoryConnectionState({
    this.connectedMemberIds = const {},
    this.pendingRequestMemberIds = const {},
    this.isLoading = false,
  });

  DirectoryConnectionState copyWith({
    Set<String>? connectedMemberIds,
    Set<String>? pendingRequestMemberIds,
    bool? isLoading,
  }) {
    return DirectoryConnectionState(
      connectedMemberIds: connectedMemberIds ?? this.connectedMemberIds,
      pendingRequestMemberIds: pendingRequestMemberIds ?? this.pendingRequestMemberIds,
```

```
      isLoading: isLoading ?? this.isLoading,
    );
  }
}

class DirectoryConnectionNotifier extends StateNotifier<DirectoryConnectionState> {
  DirectoryConnectionNotifier() : super(const DirectoryConnectionState());

  Future<void> loadUserConnections(String userId) async {
    if (userId.isEmpty) return;
    try {
      final res = await http.get(
        Uri.parse('${ApiConfig.baseUrl}/directory/connections/$userId'),
      ).timeout(const Duration(seconds: 4));
      if (res.statusCode == 200) {
        final data = jsonDecode(res.body);
        if (data['success'] == true) {
          final List connected = data['connectedUserIds'] ?? [];
          final List pending = data['pendingSentIds'] ?? [];
          state = state.copyWith(
            connectedMemberIds: connected.map((e) => e.toString()).toSet(),
            pendingRequestMemberIds: pending.map((e) => e.toString()).toSet(),
          );
        }
      }
    } catch (_) {}
  }

  Future<bool> sendFollowRequest({
    required String senderId,
    required String senderName,
    required String senderEmail,
    required Member targetMember,
  }) async {
    final memberId = targetMember.id;
    final pending = Set<String>.from(state.pendingRequestMemberIds)..add(memberId);
    state = state.copyWith(pendingRequestMemberIds: pending);

    try {
      final res = await http.post(
        Uri.parse('${ApiConfig.baseUrl}/directory/request'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'senderId': senderId,
          'senderName': senderName,
          'senderEmail': senderEmail,
          'receiverId': targetMember.id,
          'receiverName': targetMember.fullName,
```

```
        'receiverEmail': targetMember.email,
      }),
    );
    if (res.statusCode == 200 || res.statusCode == 201) {
      return true;
    }
  } catch (_) {}
  return false;
 }

 Future<bool> respondToRequest(String requestId, String status) async {
  try {
    final res = await http.post(
      Uri.parse('${ApiConfig.baseUrl}/directory/respond'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'requestId': requestId,
        'status': status,
      }),
    );
    return res.statusCode == 200;
  } catch (_) {
    return false;
  }
 }

 void acceptFollowRequest(String memberId) {
  final pending = Set<String>.from(state.pendingRequestMemberIds)..remove(memberId);
  final connected = Set<String>.from(state.connectedMemberIds)..add(memberId);
  state = state.copyWith(connectedMemberIds: connected, pendingRequestMemberIds: pending);
 }

 bool isConnected(String memberId) => state.connectedMemberIds.contains(memberId);
 bool isPending(String memberId) => state.pendingRequestMemberIds.contains(memberId);
}

final directoryConnectionProvider = StateNotifierProvider<DirectoryConnectionNotifier,
DirectoryConnectionState>((ref) {
 return DirectoryConnectionNotifier();
});
```

lib/models/user_model.dart

```
import '../api_config.dart';

class UserModel {
 final String id;
```

```
final String fullName;
final String email;
final String surname;
final String fatherName;
final String phoneNumber;
final String gender;
final String dateOfBirth;
final String nativePlace;
final String address;
final String city;
final String state;
final String maritalStatus;
final String occupation;
final String education;
final String bloodGroup;
final String profilePhoto;
final String memberId;
final String maidenName;
final String familyId;
final String familyName;
final String relationshipToHead;
final String fatherId;
final String motherId;
final String motherName;
final String paternalGrandfatherId;
final String grandfather;
final String paternalGrandmotherId;
final String grandmother;
final String maternalGrandfatherId;
final String nana;
final String maternalGrandmotherId;
final String nani;
final String spouseId;
final String spouseName;
final String familyHeadPhone;
final String role; // 'user', 'admin', 'organizer'
final bool isApproved;
final bool willingToDonateBlood;
final bool hasMatrimonialProfile;
final DateTime? createdAt;

const UserModel({
  required this.id,
  required this.fullName,
  this.email = '',
  this.surname = '',
  this.fatherName = '',
  required this.phoneNumber,
```

```
    required this.gender,
    required this.dateOfBirth,
    this.nativePlace = '',
    this.address = '',
    required this.city,
    this.state = '',
    this.maritalStatus = '',
    this.occupation = '',
    this.education = '',
    this.bloodGroup = '',
    this.profilePhoto = '',
    this.memberId = '',
    this.maidenName = '',
    this.familyId = '',
    this.familyName = '',
    this.relationshipToHead = '',
    this.fatherId = '',
    this.motherId = '',
    this.motherName = '',
    this.paternalGrandfatherId = '',
    this.grandfather = '',
    this.paternalGrandmotherId = '',
    this.grandmother = '',
    this.maternalGrandfatherId = '',
    this.nana = '',
    this.maternalGrandmotherId = '',
    this.nani = '',
    this.spouseId = '',
    this.spouseName = '',
    this.familyHeadPhone = '',
    this.role = 'user',
    this.isApproved = false,
    this.willingToDonateBlood = false,
    this.hasMatrimonialProfile = false,
    this.createdAt,
  });

  bool get isAdmin => role == 'admin' || role == 'organizer';

  factory UserModel.fromJson(Map<String, dynamic> json) {
    DateTime? parseDate(dynamic dateVal) {
      if (dateVal == null) return null;
      if (dateVal is String) return DateTime.tryParse(dateVal);
      return null;
    }

    return UserModel(
      id: json['_id'] as String? ?? json['id'] as String? ?? '',
```

```
    fullName: json['fullName'] as String? ?? '',
    email: json['email'] as String? ?? '',
    surname: json['surname'] as String? ?? '',
    fatherName: json['fatherName'] as String? ?? '',
    phoneNumber: json['phoneNumber'] as String? ?? '',
    gender: json['gender'] as String? ?? 'Male',
    dateOfBirth: json['dateOfBirth'] as String? ?? '',
    nativePlace: json['nativePlace'] as String? ?? '',
    address: json['address'] as String? ?? '',
    city: json['city'] as String? ?? '',
    state: json['state'] as String? ?? '',
    maritalStatus: json['maritalStatus'] as String? ?? '',
    occupation: json['occupation'] as String? ?? '',
    education: json['education'] as String? ?? '',
    bloodGroup: json['bloodGroup'] as String? ?? '',
    profilePhoto: (() {
      String pPhoto = json['profilePhoto'] as String? ?? '';
      if (pPhoto.startsWith('/uploads')) {
        pPhoto = '${ApiConfig.baseUrl.replaceAll('/api', '')}$pPhoto';
      }
      return pPhoto;
    })(),
    memberId: json['memberId'] as String? ?? '',
    maidenName: json['maidenName'] as String? ?? '',
    familyId: json['familyId'] as String? ?? '',
    familyName: json['familyName'] as String? ?? '',
    relationshipToHead: json['relationshipToHead'] as String? ?? 'Other',
    fatherId: json['fatherId'] as String? ?? '',
    motherId: json['motherId'] as String? ?? '',
    motherName: json['motherName'] as String? ?? '',
    paternalGrandfatherId: json['paternalGrandfatherId'] as String? ?? '',
    grandfather: json['grandfather'] as String? ?? '',
    paternalGrandmotherId: json['paternalGrandmotherId'] as String? ?? '',
    grandmother: json['grandmother'] as String? ?? '',
    maternalGrandfatherId: json['maternalGrandfatherId'] as String? ?? '',
    nana: json['nana'] as String? ?? '',
    maternalGrandmotherId: json['maternalGrandmotherId'] as String? ?? '',
    nani: json['nani'] as String? ?? '',
    spouseId: json['spouseId'] as String? ?? '',
    spouseName: json['spouseName'] as String? ?? '',
    familyHeadPhone: json['familyHeadPhone'] as String? ?? '',
    role: json['role'] as String? ?? 'user',
    isApproved: json['isApproved'] as bool? ?? false,
    willingToDonateBlood: json['willingToDonateBlood'] as bool? ?? false,
    hasMatrimonialProfile: json['hasMatrimonialProfile'] as bool? ?? false,
    createdAt: parseDate(json['createdAt']),
  );
}
```

```
Map<String, dynamic> toJson() {
  return {
    '_id': id,
    'fullName': fullName,
    'email': email,
    'surname': surname,
    'fatherName': fatherName,
    'phoneNumber': phoneNumber,
    'gender': gender,
    'dateOfBirth': dateOfBirth,
    'nativePlace': nativePlace,
    'address': address,
    'city': city,
    'state': state,
    'maritalStatus': maritalStatus,
    'occupation': occupation,
    'education': education,
    'bloodGroup': bloodGroup,
    'profilePhoto': profilePhoto,
    'memberId': memberId,
    'maidenName': maidenName,
    'familyId': familyId,
    'familyName': familyName,
    'relationshipToHead': relationshipToHead,
    'fatherId': fatherId,
    'motherId': motherId,
    'motherName': motherName,
    'paternalGrandfatherId': paternalGrandfatherId,
    'grandfather': grandfather,
    'paternalGrandmotherId': paternalGrandmotherId,
    'grandmother': grandmother,
    'maternalGrandfatherId': maternalGrandfatherId,
    'nana': nana,
    'maternalGrandmotherId': maternalGrandmotherId,
    'nani': nani,
    'spouseId': spouseId,
    'spouseName': spouseName,
    'familyHeadPhone': familyHeadPhone,
    'role': role,
    'isApproved': isApproved,
    'willingToDonateBlood': willingToDonateBlood,
    'hasMatrimonialProfile': hasMatrimonialProfile,
    'createdAt': createdAt?.toIso8601String(),
  };
}

UserModel copyWith({
```

```
    String? id,
    String? fullName,
    String? email,
    String? surname,
    String? fatherName,
    String? phoneNumber,
    String? gender,
    String? dateOfBirth,
    String? nativePlace,
    String? address,
    String? city,
    String? state,
    String? maritalStatus,
    String? occupation,
    String? education,
    String? bloodGroup,
    String? profilePhoto,
    String? memberId,
    String? maidenName,
    String? familyId,
    String? familyName,
    String? relationshipToHead,
    String? fatherId,
    String? motherId,
    String? motherName,
    String? paternalGrandfatherId,
    String? grandfather,
    String? paternalGrandmotherId,
    String? grandmother,
    String? maternalGrandfatherId,
    String? nana,
    String? maternalGrandmotherId,
    String? nani,
    String? spouseId,
    String? spouseName,
    String? familyHeadPhone,
    String? role,
    bool? isApproved,
    bool? willingToDonateBlood,
    bool? hasMatrimonialProfile,
    DateTime? createdAt,
  }) {
    return UserModel(
      id: id ?? this.id,
      fullName: fullName ?? this.fullName,
      email: email ?? this.email,
      surname: surname ?? this.surname,
      fatherName: fatherName ?? this.fatherName,
```

```
      phoneNumber: phoneNumber ?? this.phoneNumber,
      gender: gender ?? this.gender,
      dateOfBirth: dateOfBirth ?? this.dateOfBirth,
      nativePlace: nativePlace ?? this.nativePlace,
      address: address ?? this.address,
      city: city ?? this.city,
      state: state ?? this.state,
      maritalStatus: maritalStatus ?? this.maritalStatus,
      occupation: occupation ?? this.occupation,
      education: education ?? this.education,
      bloodGroup: bloodGroup ?? this.bloodGroup,
      profilePhoto: profilePhoto ?? this.profilePhoto,
      memberId: memberId ?? this.memberId,
      maidenName: maidenName ?? this.maidenName,
      familyId: familyId ?? this.familyId,
      familyName: familyName ?? this.familyName,
      relationshipToHead: relationshipToHead ?? this.relationshipToHead,
      fatherId: fatherId ?? this.fatherId,
      motherId: motherId ?? this.motherId,
      motherName: motherName ?? this.motherName,
      paternalGrandfatherId: paternalGrandfatherId ?? this.paternalGrandfatherId,
      grandfather: grandfather ?? this.grandfather,
      paternalGrandmotherId: paternalGrandmotherId ?? this.paternalGrandmotherId,
      grandmother: grandmother ?? this.grandmother,
      maternalGrandfatherId: maternalGrandfatherId ?? this.maternalGrandfatherId,
      nana: nana ?? this.nana,
      maternalGrandmotherId: maternalGrandmotherId ?? this.maternalGrandmotherId,
      nani: nani ?? this.nani,
      spouseId: spouseId ?? this.spouseId,
      spouseName: spouseName ?? this.spouseName,
      familyHeadPhone: familyHeadPhone ?? this.familyHeadPhone,
      role: role ?? this.role,
      isApproved: isApproved ?? this.isApproved,
      willingToDonateBlood: willingToDonateBlood ?? this.willingToDonateBlood,
      hasMatrimonialProfile: hasMatrimonialProfile ?? this.hasMatrimonialProfile,
      createdAt: createdAt ?? this.createdAt,
    );
  }
}
```

lib/models/member_model.dart

```
class Member {
  final String id;
  final String fullName;
  final String initials;
  final String gender;
  final String mobileNumber;
```

```
final String email;
final String village;
final String city;
final String district;
final String state;
final String profession;
final String company;
final String education;
final String bloodGroup;
final int age;
final String maritalStatus;
final String businessCategory;
final List<String> skills;
final List<String> languages;
final String avatarUrl;
final String joinedDate;
final bool isVerified;
final bool isActive;

const Member({
  required this.id,
  required this.fullName,
  required this.initials,
  required this.gender,
  required this.mobileNumber,
  required this.email,
  required this.village,
  required this.city,
  required this.district,
  required this.state,
  required this.profession,
  required this.company,
  required this.education,
  required this.bloodGroup,
  required this.age,
  required this.maritalStatus,
  required this.businessCategory,
  required this.skills,
  required this.languages,
  required this.avatarUrl,
  required this.joinedDate,
  required this.isVerified,
  required this.isActive,
});

String get firstLetter {
  if (fullName.isEmpty) return '#';
  final char = fullName.trim()[0].toUpperCase();
```

```
  final RegExp alpha = RegExp(r'[A-Z]');
  return alpha.hasMatch(char) ? char : '#';
}

String get fullLocation => '$village, $city';
String get fullAddress => '$village, $city, Dist. $district, $state';

factory Member.fromJson(Map<String, dynamic> json) {
  return Member(
    id: json['id'] as String? ?? '',
    fullName: json['fullName'] as String? ?? '',
    initials: json['initials'] as String? ?? '',
    gender: json['gender'] as String? ?? 'Male',
    mobileNumber: json['mobileNumber'] as String? ?? '',
    email: json['email'] as String? ?? '',
    village: json['village'] as String? ?? '',
    city: json['city'] as String? ?? '',
    district: json['district'] as String? ?? '',
    state: json['state'] as String? ?? 'Gujarat',
    profession: json['profession'] as String? ?? '',
    company: json['company'] as String? ?? '',
    education: json['education'] as String? ?? '',
    bloodGroup: json['bloodGroup'] as String? ?? '',
    age: (json['age'] as num?)?.toInt() ?? 0,
    maritalStatus: json['maritalStatus'] as String? ?? 'Married',
    businessCategory: json['businessCategory'] as String? ?? '',
    skills: (json['skills'] as List<dynamic>?)
          ?.map((e) => e.toString())
          .toList() ??
        const [],
    languages: (json['languages'] as List<dynamic>?)
          ?.map((e) => e.toString())
          .toList() ??
        const [],
    avatarUrl: json['avatarUrl'] as String? ?? '',
    joinedDate: json['joinedDate'] as String? ?? '',
    isVerified: json['isVerified'] as bool? ?? false,
    isActive: json['isActive'] as bool? ?? true,
  );
}

Map<String, dynamic> toJson() {
  return {
    'id': id,
    'fullName': fullName,
    'initials': initials,
    'gender': gender,
    'mobileNumber': mobileNumber,
```

```
    'email': email,
    'village': village,
    'city': city,
    'district': district,
    'state': state,
    'profession': profession,
    'company': company,
    'education': education,
    'bloodGroup': bloodGroup,
    'age': age,
    'maritalStatus': maritalStatus,
    'businessCategory': businessCategory,
    'skills': skills,
    'languages': languages,
    'avatarUrl': avatarUrl,
    'joinedDate': joinedDate,
    'isVerified': isVerified,
    'isActive': isActive,
  };
}

Member copyWith({
  String? id,
  String? fullName,
  String? initials,
  String? gender,
  String? mobileNumber,
  String? email,
  String? village,
  String? city,
  String? district,
  String? state,
  String? profession,
  String? company,
  String? education,
  String? bloodGroup,
  int? age,
  String? maritalStatus,
  String? businessCategory,
  List<String>? skills,
  List<String>? languages,
  String? avatarUrl,
  String? joinedDate,
  bool? isVerified,
  bool? isActive,
}) {
  return Member(
    id: id ?? this.id,
```

```
      fullName: fullName ?? this.fullName,
      initials: initials ?? this.initials,
      gender: gender ?? this.gender,
      mobileNumber: mobileNumber ?? this.mobileNumber,
      email: email ?? this.email,
      village: village ?? this.village,
      city: city ?? this.city,
      district: district ?? this.district,
      state: state ?? this.state,
      profession: profession ?? this.profession,
      company: company ?? this.company,
      education: education ?? this.education,
      bloodGroup: bloodGroup ?? this.bloodGroup,
      age: age ?? this.age,
      maritalStatus: maritalStatus ?? this.maritalStatus,
      businessCategory: businessCategory ?? this.businessCategory,
      skills: skills ?? this.skills,
      languages: languages ?? this.languages,
      avatarUrl: avatarUrl ?? this.avatarUrl,
      joinedDate: joinedDate ?? this.joinedDate,
      isVerified: isVerified ?? this.isVerified,
      isActive: isActive ?? this.isActive,
    );
  }

  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is Member && runtimeType == other.runtimeType && id == other.id;

  @override
  int get hashCode => id.hashCode;
}
```

lib/screens/member_directory_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../constants/app_colors.dart';
import '../constants/app_constants.dart';
import '../providers/member_providers.dart';
import '../providers/theme_provider.dart';
import '../providers/auth_provider.dart';
import '../widgets/filter_chips_bar.dart';
import '../widgets/filter_modal_sheet.dart';
import '../widgets/member_card.dart';
import '../widgets/search_bar_widget.dart';
```

```
import '../widgets/sort_modal_sheet.dart';
import '../widgets/sticky_alphabet_index.dart';
import '../widgets/app_bottom_nav_bar.dart';

class MemberDirectoryScreen extends ConsumerStatefulWidget {
  const MemberDirectoryScreen({super.key});

  @override
  ConsumerState<MemberDirectoryScreen> createState() =>
      _MemberDirectoryScreenState();
}

class _MemberDirectoryScreenState
    extends ConsumerState<MemberDirectoryScreen> {
  final ScrollController _scrollController = ScrollController();
  final Map<String, GlobalKey> _letterKeys = {};
  String _activeLetter = 'A';

  @override
  void initState() {
    super.initState();
    for (final letter in AppConstants.alphabetIndex) {
      _letterKeys[letter] = GlobalKey();
    }
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final user = ref.read(currentUserProvider);
      if (user != null) {
        ref.read(directoryConnectionProvider.notifier).loadUserConnections(user.id.isNotEmpty ? user.id :
user.email);
      }
    });
  }

  @override
  void dispose() {
    _scrollController.dispose();
    super.dispose();
  }

  void _scrollToLetter(String letter) {
    setState(() {
      _activeLetter = letter;
    });

    final key = _letterKeys[letter];
    if (key != null && key.currentContext != null) {
      Scrollable.ensureVisible(
        key.currentContext!,
```

```
      duration: const Duration(milliseconds: 400),
      curve: Curves.easeInOutCubic,
    );
  }
}

void _openFilterModal([String category = 'Village']) {
  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    backgroundColor: Colors.transparent,
    builder: (context) => FilterModalSheet(initialCategory: category),
  );
}

void _openSortModal() {
  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.transparent,
    builder: (context) => const SortModalSheet(),
  );
}

@override
Widget build(BuildContext context) {
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final filteredAsync = ref.watch(filteredMembersProvider);
  final groupedAsync = ref.watch(groupedMembersProvider);
  final rawCount = ref.watch(rawMemberListProvider).asData?.value.length ?? 0;
  final sortOption = ref.watch(sortOptionProvider);

  return Scaffold(
    body: SafeArea(
      top: false,
      child: Column(
        children: [
          // 1. Premium Blue Gradient Header
          Container(
            padding: EdgeInsets.only(
              top: MediaQuery.of(context).padding.top + 16,
              bottom: 20,
              left: 20,
              right: 16,
            ),
            decoration: BoxDecoration(
              gradient: isDark
                  ? AppColors.headerGradientDark
                  : AppColors.headerGradientLight,
```

```
  boxShadow: [
    BoxShadow(
      color: AppColors.primaryBlue.withValues(alpha: 0.3),
      blurRadius: 15,
      offset: const Offset(0, 5),
    ),
  ],
  borderRadius: const BorderRadius.vertical(
    bottom: Radius.circular(24),
  ),
),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Expanded(
          child: Row(
            children: [
              IconButton(
                icon: const Icon(Icons.arrow_back_rounded, color: Colors.white, size: 20),
                onPressed: () => context.pop(),
                constraints: const BoxConstraints(),
                padding: const EdgeInsets.only(right: 8),
                tooltip: 'Back to Home',
              ),
              // Subtitle tag
              Flexible(
                child: Container(
                  padding: const EdgeInsets.symmetric(
                      horizontal: 10, vertical: 4),
                  decoration: BoxDecoration(
                    color: Colors.white.withValues(alpha: 0.15),
                    borderRadius: BorderRadius.circular(20),
                  ),
                  child: const Row(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      Icon(Icons.groups_rounded,
                          color: Colors.white, size: 14),
                      SizedBox(width: 4),
                      Flexible(
                        child: Text(
                          'Babriyawad Darji Samaj',
                          style: TextStyle(
                            color: Colors.white,
                            fontSize: 11,
```

```
                    fontWeight: FontWeight.w600,
                    letterSpacing: 0.5,
                  ),
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                ),
              ),
            ],
          ),
        ),
      ),
    ),

    // Light/Dark Theme Toggle
    IconButton(
      onPressed: () {
        ref.read(themeModeProvider.notifier).toggleTheme();
      },
      icon: Icon(
        isDark ? Icons.light_mode : Icons.dark_mode,
        color: Colors.white,
      ),
      tooltip: 'Toggle Theme',
    ),
  ],
),
const SizedBox(height: 10),

// Main App Title & Member Counter
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  crossAxisAlignment: CrossAxisAlignment.end,
  children: [
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            AppConstants.appName,
            style: const TextStyle(
              fontSize: 26,
              fontWeight: FontWeight.bold,
              color: Colors.white,
              letterSpacing: -0.5,
            ),
            maxLines: 1,
```

```
                overflow: TextOverflow.ellipsis,
              ),
              const Text(
                AppConstants.appSubTitle,
                style: TextStyle(
                  fontSize: 14,
                  color: Colors.white70,
                  fontWeight: FontWeight.w400,
                ),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
              ),
            ],
          ),
        ),
        const SizedBox(width: 12),

        // Member Count Badge
        Container(
          padding: const EdgeInsets.symmetric(
              horizontal: 12, vertical: 6),
          decoration: BoxDecoration(
            color: Colors.white.withValues(alpha: 0.2),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(
              color: Colors.white.withValues(alpha: 0.3),
              width: 1,
            ),
          ),
          child: Text(
            '$rawCount Members',
            style: const TextStyle(
              color: Colors.white,
              fontSize: 12,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      ],
    ),
  ],
 ),
),

// 2. Debounced Search Bar
SearchBarWidget(
  onOpenFilter: () => _openFilterModal(),
  onOpenSort: _openSortModal,
```

```
),

// 3. Filter Chips Bar
FilterChipsBar(
  onSelectCategoryFilter: (category) => _openFilterModal(category),
),

// 4. Main Directory List View with Sticky A-Z Index
Expanded(
  child: filteredAsync.when(
    loading: () => const Center(
      child: CircularProgressIndicator(color: AppColors.accentBlue),
    ),
    error: (err, stack) => Center(
      child: Text('Error loading directory: $err'),
    ),
    data: (members) {
      if (members.isEmpty) {
        return _buildEmptyState(context);
      }

      // If standard A-Z sort, show alphabetical headers and sticky index
      final bool isAZSort = sortOption == SortOption.nameAZ;

      return Stack(
        children: [
          // List of Members
          groupedAsync.when(
            data: (groupedMap) {
              if (!isAZSort) {
                // Standard flat list when sorted by Profession, City, Date, etc.
                return ListView.builder(
                  controller: _scrollController,
                  padding: const EdgeInsets.only(
                      bottom: 24, left: 4, right: 32),
                  itemCount: members.length,
                  itemBuilder: (context, index) {
                    final member = members[index];
                    return MemberCard(
                      member: member,
                      onTap: () => context.push(
                        '/member/${member.id}',
                        extra: member,
                      ),
                    );
                  },
                );
              }
```

```
// Grouped A-Z List with Headers
final letters = groupedMap.keys.toList()..sort();

return ListView(
  controller: _scrollController,
  padding: const EdgeInsets.only(
      bottom: 24, left: 4, right: 32),
  children: letters.map((letter) {
    final letterMembers = groupedMap[letter] ?? [];

    return Column(
      key: _letterKeys[letter],
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // Section Header Letter Badge
        Padding(
          padding: const EdgeInsets.only(
              left: 16, top: 12, bottom: 6),
          child: Row(
            children: [
              Container(
                padding: const EdgeInsets.symmetric(
                    horizontal: 10, vertical: 4),
                decoration: BoxDecoration(
                  color: AppColors.accentBlue,
                  borderRadius:
                      BorderRadius.circular(8),
                ),
                child: Text(
                  letter,
                  style: const TextStyle(
                    color: Colors.white,
                    fontSize: 14,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: Divider(
                  color: isDark
                      ? AppColors.borderDark
                      : AppColors.borderLight,
                ),
              ),
              Text(
                '${letterMembers.length}',
```

```
                  style: TextStyle(
                    fontSize: 12,
                    color: isDark
                        ? AppColors.textSecondaryDark
                        : AppColors.textSecondaryLight,
                  ),
                ),
                const SizedBox(width: 16),
              ],
            ),
          ),

          // Members under this letter
          ...letterMembers.map(
            (member) => MemberCard(
              member: member,
              onTap: () => context.push(
                '/member/${member.id}',
                extra: member,
              ),
            ),
          ),
        ],
      );
    }).toList(),
  );
 },
 loading: () => const Center(
  child: CircularProgressIndicator(),
 ),
 error: (_, __) => const SizedBox(),
),

// Right-side Sticky A-Z Alphabet Index Bar
if (isAZSort)
 Positioned(
  right: 0,
  top: 0,
  bottom: 0,
  child: Align(
   alignment: Alignment.centerRight,
   child: StickyAlphabetIndex(
    availableLetters: groupedAsync.asData?.value.keys.toList() ?? [],
    activeLetter: _activeLetter,
    onLetterTap: _scrollToLetter,
   ),
  ),
 ),
```

```
                ],
              );
            },
          ),
        ),
      ],
    ),
  ),
  bottomNavigationBar: const AppBottomNavBar(currentRoute: '/'),
 );
}

Widget _buildEmptyState(BuildContext context) {
 final isDark = Theme.of(context).brightness == Brightness.dark;
 return Center(
  child: Padding(
   padding: const EdgeInsets.all(32.0),
   child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
     Icon(
      Icons.search_off_rounded,
      size: 64,
      color: isDark
        ? AppColors.textSecondaryDark
        : AppColors.textSecondaryLight,
     ),
     const SizedBox(height: 16),
     Text(
      'No Members Found',
      style: Theme.of(context).textTheme.titleLarge?.copyWith(
        fontWeight: FontWeight.bold,
       ),
     ),
     const SizedBox(height: 8),
     Text(
      'No member profiles match your current search query or applied filter options.',
      textAlign: TextAlign.center,
      style: TextStyle(
       fontSize: 14,
       color: isDark
         ? AppColors.textSecondaryDark
         : AppColors.textSecondaryLight,
      ),
     ),
     const SizedBox(height: 20),
     OutlinedButton.icon(
      onPressed: () {
```

```
                ref.read(searchQueryProvider.notifier).state = '';
                ref.read(activeFiltersProvider.notifier).clearAll();
              },
              icon: const Icon(Icons.refresh),
              label: const Text('Reset All Filters'),
              style: OutlinedButton.styleFrom(
                foregroundColor: AppColors.accentBlue,
                side: const BorderSide(color: AppColors.accentBlue),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

backend/models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  fullName: {
    type: String,
    required: true,
    trim: true,
  },
  email: {
    type: String,
    trim: true,
    lowercase: true,
    index: { unique: true, sparse: true },
  },
  phoneNumber: {
    type: String,
    trim: true,
    index: { unique: true, sparse: true },
  },
  role: {
    type: String,
    enum: ['user', 'admin', 'organizer'],
    default: 'user',
  },
  isApproved: {
```

```
    type: Boolean,
    default: false,
  },
  createdAt: {
    type: Date,
    default: Date.now,
  },
});

const User = mongoose.model('User', userSchema, 'users');
const DirectoryUser = mongoose.model('DirectoryUser', userSchema, 'directory');
User.DirectoryUser = DirectoryUser;

module.exports = User;
```

backend/models/Member.js

```
const mongoose = require('mongoose');

const memberSchema = new mongoose.Schema({
  memberId: {
    type: String,
    required: true,
    unique: true,
    index: true,
    trim: true,
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null,
    index: true,
  },
  fullName: {
    type: String,
    required: true,
    trim: true,
    index: true,
  },
  firstName: {
    type: String,
    default: '',
    trim: true,
  },
  middleName: {
    type: String,
    default: '',
```

```
  trim: true,
},
lastName: {
  type: String,
  default: '',
  trim: true,
},
maidenName: {
  type: String,
  default: '',
  trim: true,
  index: true,
},
gender: {
  type: String,
  enum: ['Male', 'Female', 'Other'],
  default: 'Male',
},
dateOfBirth: {
  type: String,
  default: '',
},
phoneNumber: {
  type: String,
  default: '',
  trim: true,
},
email: {
  type: String,
  default: '',
  trim: true,
},
city: {
  type: String,
  default: '',
  trim: true,
},
village: {
  type: String,
  default: '',
  trim: true,
},
state: {
  type: String,
  default: 'Gujarat',
  trim: true,
},
address: {
```

```
  type: String,
  default: '',
  trim: true,
},
age: {
  type: String,
  default: '',
  trim: true,
},
profilePhoto: {
  type: String,
  default: '',
},
fatherId: {
  type: String,
  default: '',
  trim: true,
},
fatherName: {
  type: String,
  default: '',
  trim: true,
},
motherId: {
  type: String,
  default: '',
  trim: true,
},
motherName: {
  type: String,
  default: '',
  trim: true,
},
paternalGrandfatherId: {
  type: String,
  default: '',
  trim: true,
},
grandfather: {
  type: String,
  default: '',
  trim: true,
},
paternalGrandmotherId: {
  type: String,
  default: '',
  trim: true,
},
```

```
grandmother: {
  type: String,
  default: '',
  trim: true,
},
maternalGrandfatherId: {
  type: String,
  default: '',
  trim: true,
},
nana: {
  type: String,
  default: '',
  trim: true,
},
maternalGrandmotherId: {
  type: String,
  default: '',
  trim: true,
},
nani: {
  type: String,
  default: '',
  trim: true,
},
spouseId: {
  type: String,
  default: '',
  trim: true,
},
spouseName: {
  type: String,
  default: '',
  trim: true,
},
familyId: {
  type: String,
  default: '',
  trim: true,
  index: true,
},
relationshipToHead: {
  type: String,
  default: 'Self',
},
isDeceased: {
  type: Boolean,
  default: false,
```

```
  },
  hasMatrimonialProfile: {
    type: Boolean,
    default: false,
  },
  matrimonialVisibility: {
    type: Boolean,
    default: true,
  },
}, {
  timestamps: true,
});

module.exports = mongoose.model('Member', memberSchema);
```

backend/models/MatrimonialProfile.js

```
const mongoose = require('mongoose');

const matrimonialProfileSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    unique: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  gender: {
    type: String,
    required: true,
    trim: true
  },
  dob: {
    type: Date,
    required: true
  },
  heightCm: {
    type: Number,
    required: true
  },
  weightKg: {
    type: Number,
    required: true
```

```
},
bloodGroup: {
  type: String,
  trim: true
},
maritalStatus: {
  type: String,
  trim: true
},
education: {
  type: String,
  trim: true
},
occupation: {
  type: String,
  trim: true
},
company: {
  type: String,
  trim: true
},
annualIncome: {
  type: Number,
  default: 0
},
village: {
  type: String,
  trim: true
},
city: {
  type: String,
  trim: true
},
workingCountry: {
  type: String,
  trim: true,
  default: ''
},
description: {
  type: String,
  trim: true,
  default: ''
},
partnerExpectations: {
  type: String,
  trim: true,
  default: ''
},
```

```
partnerExpectationsHobbies: {
  type: [String],
  default: []
},
additionalPhotos: {
  type: [String],
  default: []
},
socialLinks: {
  showSocialLinks: { type: Boolean, default: false },
  instagramUrl: { type: String, trim: true, default: '' },
  facebookUrl: { type: String, trim: true, default: '' }
},
familyInformation: {
  fatherName: { type: String, trim: true, default: '' },
  motherName: { type: String, trim: true, default: '' },
  grandfather: { type: String, trim: true, default: '' },
  grandmother: { type: String, trim: true, default: '' },
  nana: { type: String, trim: true, default: '' },
  nani: { type: String, trim: true, default: '' },
  familyOccupation: { type: String, trim: true, default: '' }
},
lifestyle: {
  languages: { type: [String], default: [] },
  hobbies: { type: [String], default: [] },
  diet: { type: String, trim: true, default: 'Vegetarian' },
  smoking: { type: String, trim: true, default: 'No' },
  drinking: { type: String, trim: true, default: 'No' }
},
partnerPreferences: {
  ageMin: { type: Number, default: 18 },
  ageMax: { type: Number, default: 60 },
  heightMin: { type: Number, default: 120 },
  heightMax: { type: Number, default: 220 },
  education: { type: String, trim: true, default: '' },
  occupation: { type: String, trim: true, default: '' },
  city: { type: String, trim: true, default: '' },
  village: { type: String, trim: true, default: '' }
},
visibilitySettings: {
  showPhone: { type: Boolean, default: false },
  showAddress: { type: Boolean, default: false },
  showEmail: { type: Boolean, default: false }
},
profilePhoto: {
  type: String,
  trim: true,
  default: ''
```

```
  },
  introductionVideo: {
    type: String,
    trim: true,
    default: ''
  },
  memberId: {
    type: String,
    trim: true,
    default: '',
    index: true,
  },
  hasMatrimonialProfile: {
    type: Boolean,
    default: true,
  },
  visibility: {
    type: Boolean,
    default: true,
  },
  profileStatus: {
    type: String,
    enum: ['Pending', 'Approved', 'Rejected'],
    default: 'Approved'
  },
  createdDate: {
    type: Date,
    default: Date.now
  },
  updatedDate: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('MatrimonialProfile', matrimonialProfileSchema,
'matrimonial_profiles');
```

backend/models/Campaign.js

```
const mongoose = require('mongoose');

const dynamicFieldSchema = new mongoose.Schema({
  fieldName: { type: String, required: true, trim: true },
  label: { type: String, required: true, trim: true },
  type: {
    type: String,
```

```
    enum: ['text', 'number', 'dropdown', 'radio', 'checkbox', 'date'],
    default: 'text',
  },
  required: { type: Boolean, default: false },
  options: [{ type: String, trim: true }],
  validationRules: { type: String, default: '' },
  order: { type: Number, default: 0 },
});

const campaignSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Campaign title is required'],
      trim: true,
    },
    description: {
      type: String,
      required: [true, 'Campaign description is required'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Campaign category is required'],
      trim: true,
    },
    bannerUrl: {
      type: String,
      default: '',
      trim: true,
    },
    startDate: {
      type: Date,
      required: [true, 'Start date is required'],
    },
    endDate: {
      type: Date,
      required: [true, 'End date is required'],
    },
    status: {
      type: String,
      enum: ['Draft', 'Upcoming', 'Active', 'Completed', 'Cancelled'],
      default: 'Active',
    },
    targetAmount: {
      type: Number,
      default: 0,
      min: [0, 'Target amount cannot be negative'],
```

```
    },
    amountRaised: {
      type: Number,
      default: 0,
      min: [0, 'Amount raised cannot be negative'],
    },
    objective: {
      type: String,
      default: '',
      trim: true,
    },
    contactInfo: {
      phone: { type: String, default: '', trim: true },
      email: { type: String, default: '', trim: true },
      organizerName: { type: String, default: '', trim: true },
    },
    additionalNotes: {
      type: String,
      default: '',
      trim: true,
    },
    location: {
      type: String,
      default: '',
      trim: true,
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: false,
    },
    dynamicFields: [dynamicFieldSchema],
  },
  {
    timestamps: true,
  }
);

// Virtual property to dynamically compute effective status if not set to Cancelled or Draft
campaignSchema.methods.getEffectiveStatus = function () {
  if (this.status === 'Cancelled' || this.status === 'Draft') {
    return this.status;
  }
  const now = new Date();
  if (this.startDate > now) {
    return 'Upcoming';
  } else if (this.endDate < now) {
    return 'Completed';
```

```
  } else {
    return 'Active';
  }
};

module.exports = mongoose.model('Campaign', campaignSchema, 'campaigns');
```

backend/models/Post.js

```
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
  authorName: {
    type: String,
    required: true,
  },
  avatarText: {
    type: String,
    default: '',
  },
  avatarColor: {
    type: String,
    default: '#F57C00', // Saffron hex
  },
  content: {
    type: String,
    required: true,
  },
  mediaUrl: {
    type: String,
    default: '',
  },
  likes: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
    },
  ],
  comments: [
    {
      userId: {
        type: mongoose.Schema.Types.ObjectId,
```

```
      ref: 'User',
      required: true,
    },
    authorName: {
      type: String,
      required: true,
    },
    content: {
      type: String,
      required: true,
    },
    likes: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
      }
    ],
    replies: [
      {
        userId: {
          type: mongoose.Schema.Types.ObjectId,
          ref: 'User',
          required: true,
        },
        authorName: {
          type: String,
          required: true,
        },
        content: {
          type: String,
          required: true,
        },
        createdAt: {
          type: Date,
          default: Date.now,
        }
      }
    ],
    createdAt: {
      type: Date,
      default: Date.now,
    },
  },
],
sharesCount: {
  type: Number,
  default: 0,
},
```

```
  createdAt: {
    type: Date,
    default: Date.now,
  },
});

module.exports = mongoose.model('Post', postSchema, 'posts');
```

4.3 Outputs

Login Page

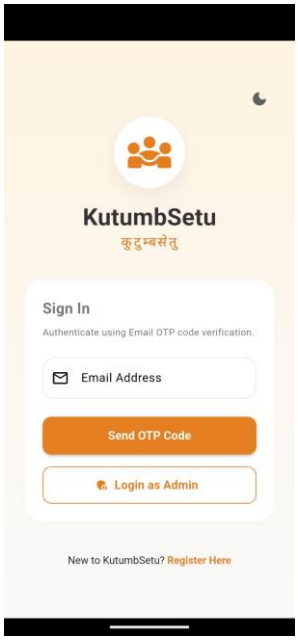


Fig 4.3.1 Login Page

Registration Page

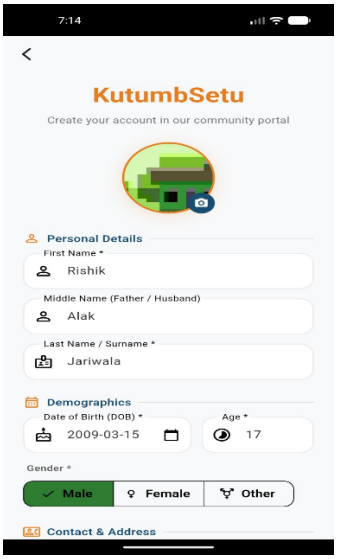


Fig 4.3.2 Registration Page(1)

Registration Page

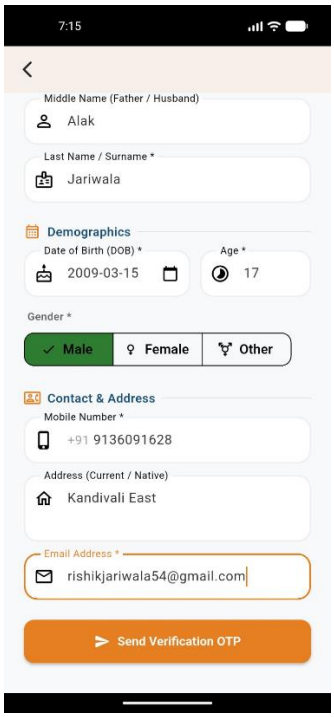


Fig 4.3.3 Registration Page(2)

Home Page

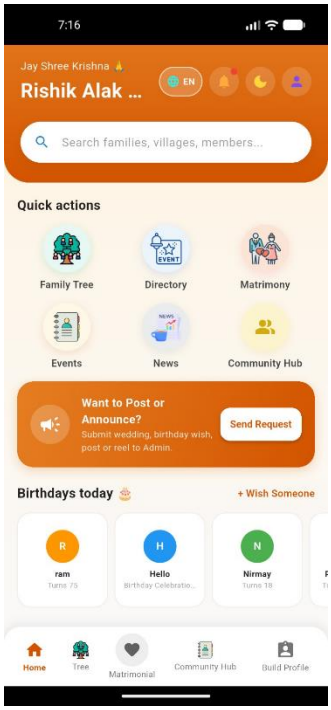


Fig 4.3.4 Home Page

Home Page

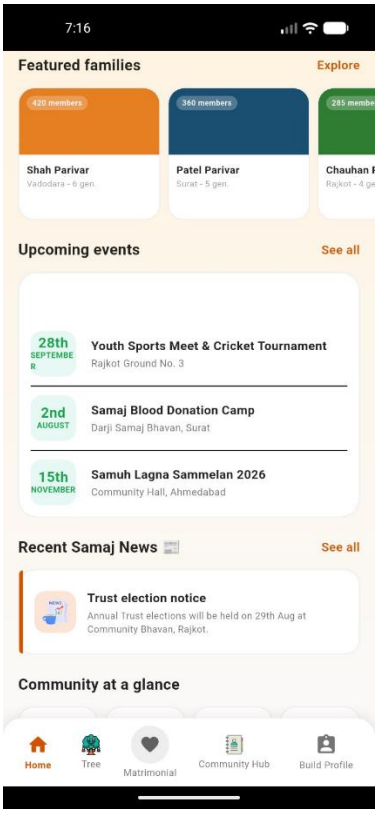


Fig 4.3.5 Home Page(2)

Home Page

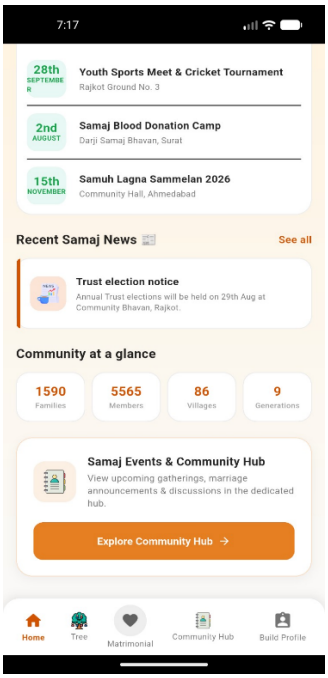


Fig 4.3.6 Home Page(3)

Family Tree

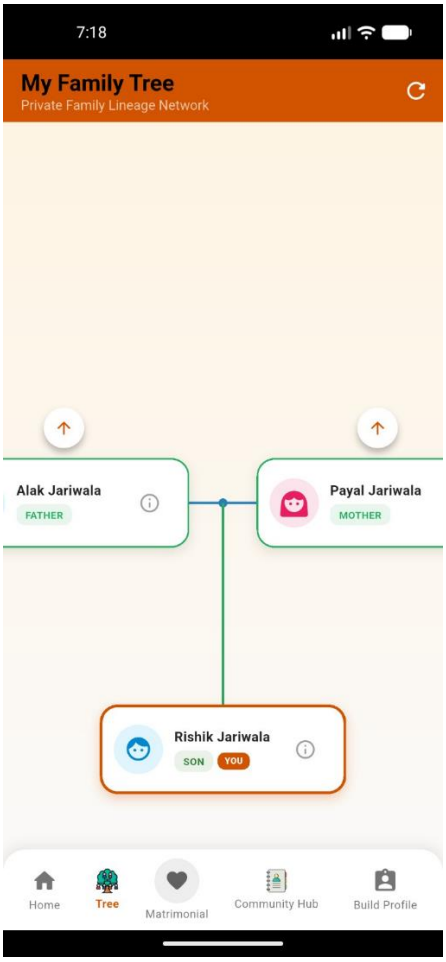


Fig 4.3.7 Family Tree

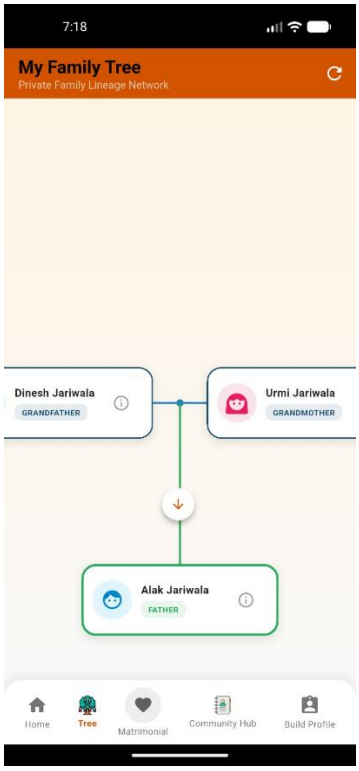


Fig 4.3.8 Paternal Family Tree

Family Tree

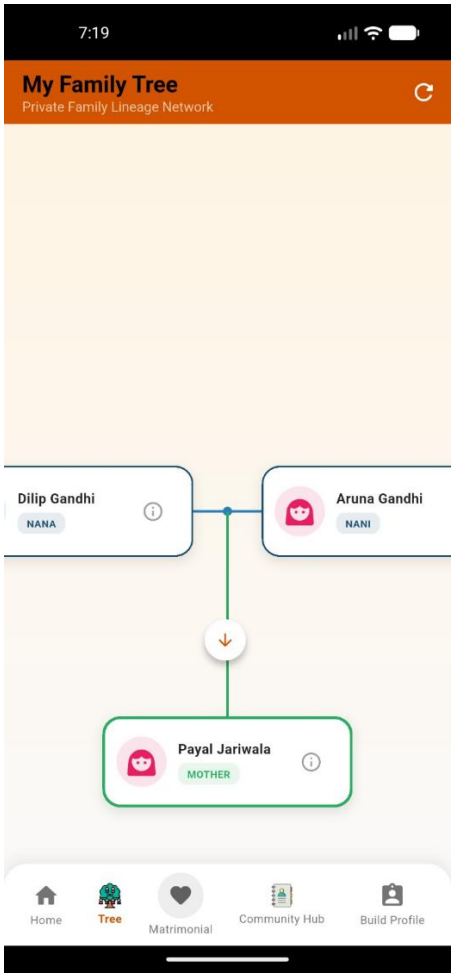


Fig 4.3.9 Maternal Family Tree

Matrimonial Page

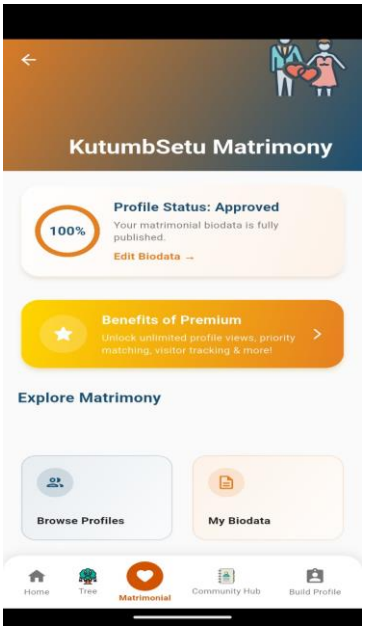


Fig 4.4.0 Matrimonial Tree

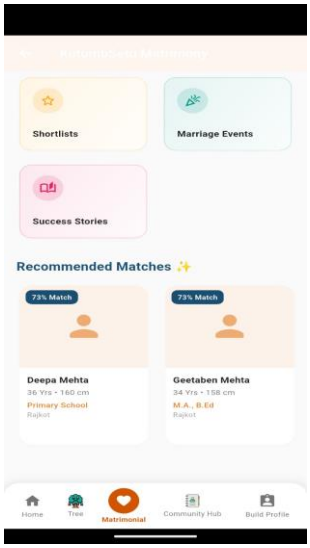


Fig 4.4.1 Matrimonial Page

Directory Page

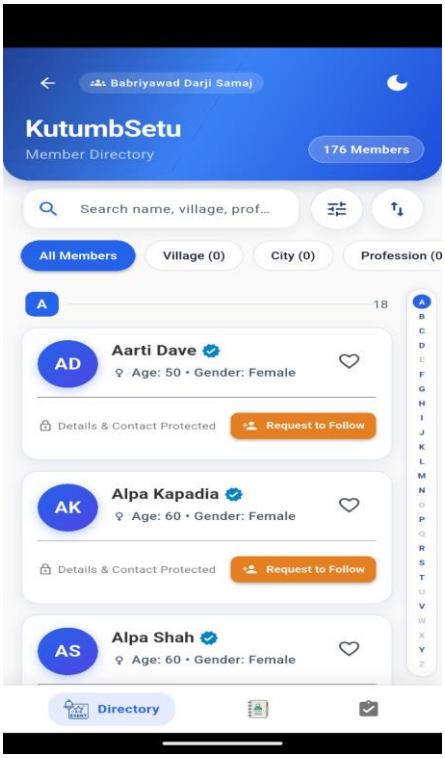


Fig 4.4.2 Directory Page

Events Page

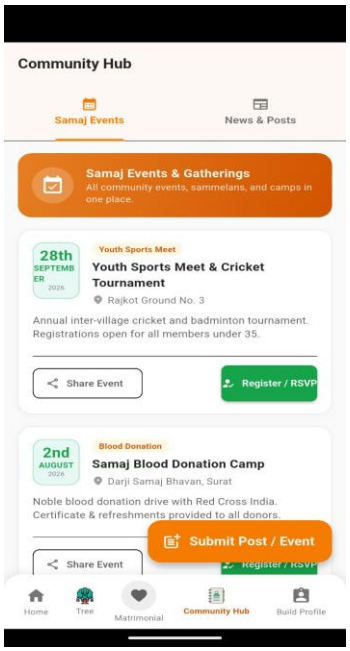


Fig 4.4.3 Events Page

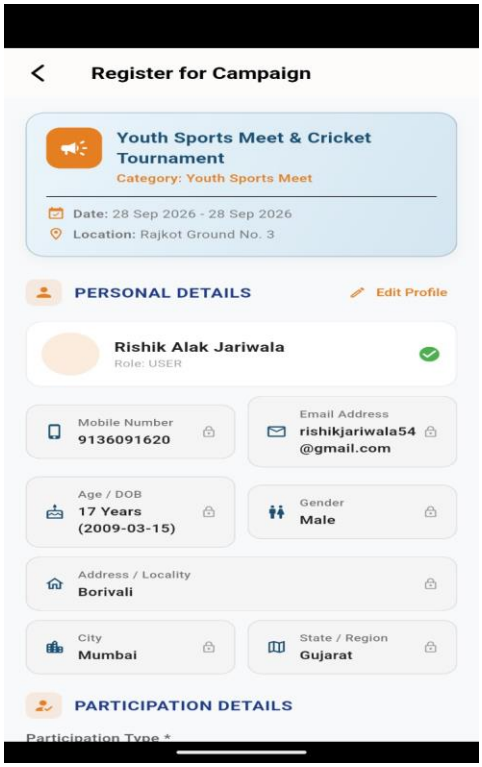


Fig 4.4.4 Register For Event

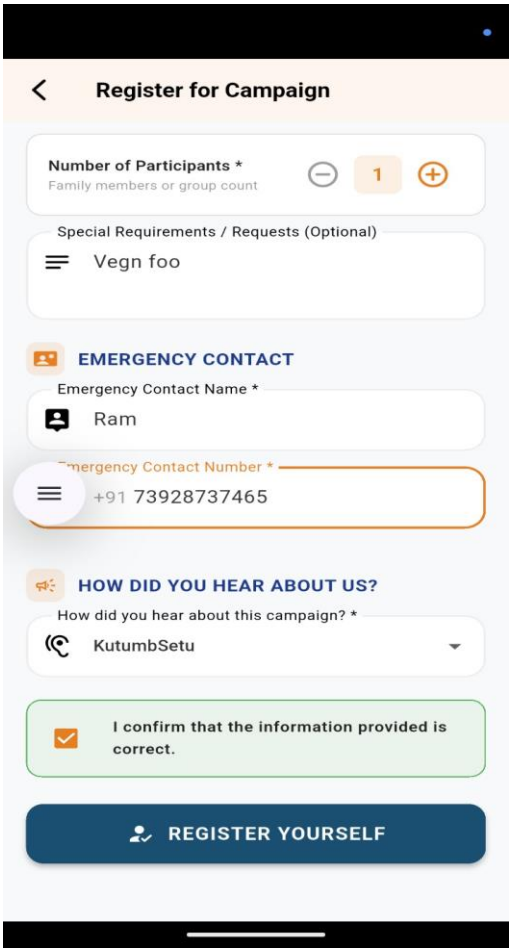


Fig 4.4.5 Register for Campaign

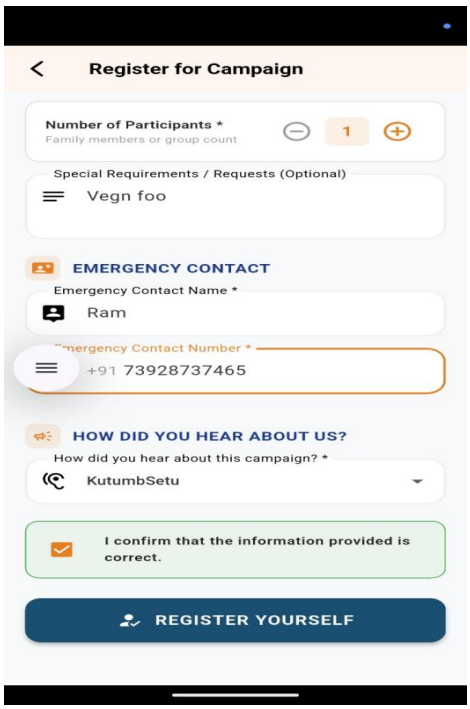


Fig 4.4.5 Register for Campaign

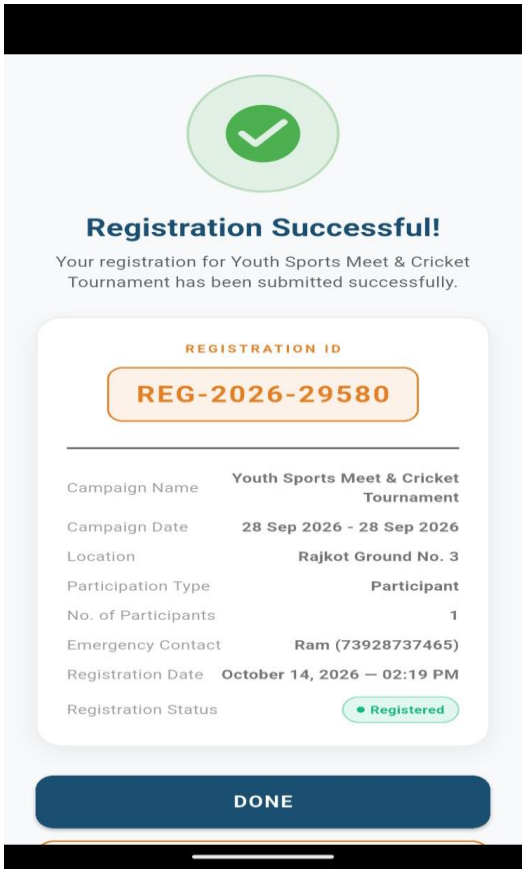


Fig 4.4.6 Events Page and Campaign Registrations Success

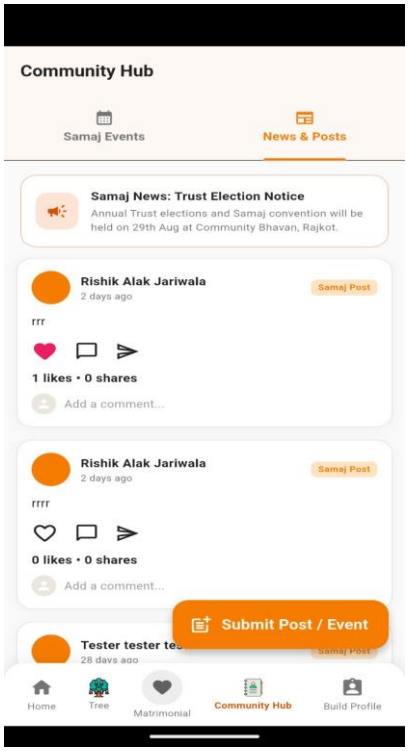


Fig 4.4.7 Community page for posts like,comment,share

Admin page

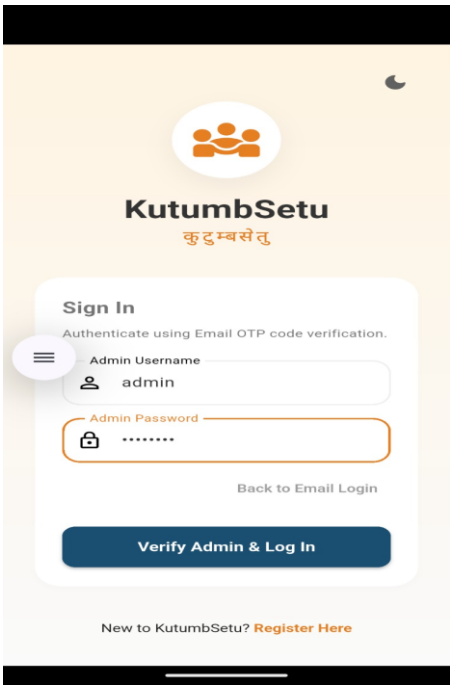


Fig 4.4.8 Admin login page

Admin panel functionalities

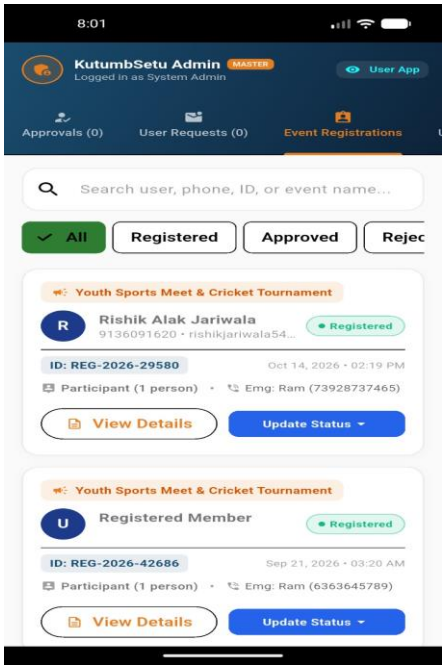


Fig 4.4.9 Admin page for event registration

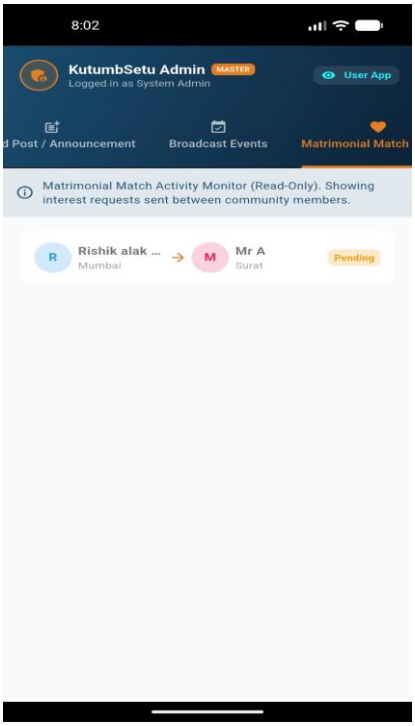


Fig 4.5.0 Admin page for Matrimonial

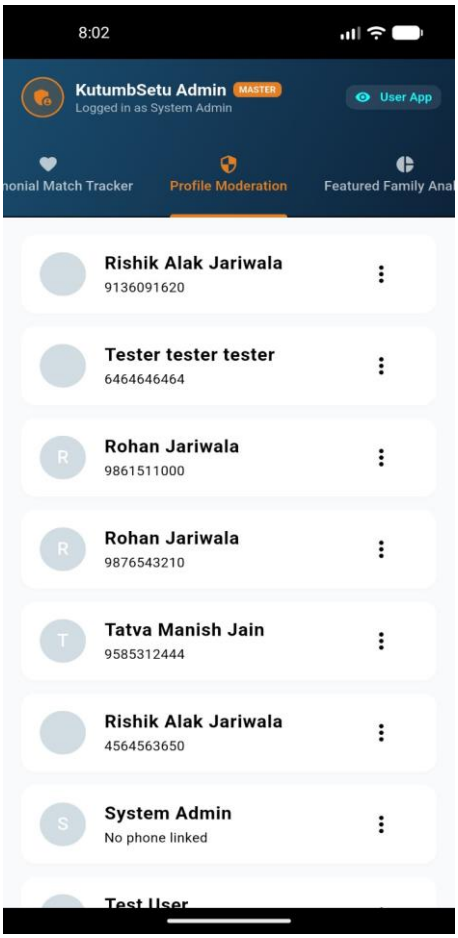


Fig 4.5.1 Admin page for directories

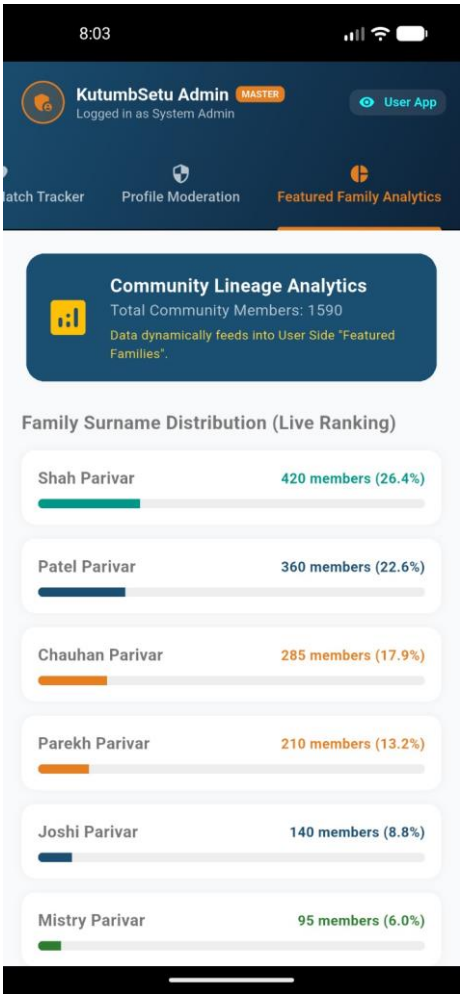


Fig 4.4.9 Admin page for featured families

Chapter 5

5.1 Advantages

1. Digital Preservation of Dual-Lineage Family Heritage: Unlike traditional paper-based community directories (*Vasti Patrak*) that degrade over time, KutumbSetu digitizes multi-generational family trees on an interactive canvas. It uniquely tracks both Paternal and Maternal lineages with smart relation-linking algorithms. This ensures that younger generations remain connected to their ancestral roots, native villages, and cultural identity regardless of geographic dispersal.

2. Privacy-First Matrimonial Ecosystem with Two-Way Consent: Traditional matrimonial biodata distribution via circulating paper booklets or unmoderated WhatsApp groups exposes sensitive personal information to privacy risks. KutumbSetu resolves this by implementing a secure two-tier consent gatekeeper where candidate biodatas are browsable within the verified community, while direct contact numbers and WhatsApp links remain strictly masked until both parties mutually accept a match request.

3. Rapid Member Discovery with Multi-Parametric Filtering: The application eliminates the friction of manually flipping through voluminous physical registers by providing real-time search across thousands of records. Members can instantly filter community peers by Native Village, Surname, Current City, Blood Group, or Profession. The addition of a sticky A–Z alphabetical index scroller and 1-tap native call/WhatsApp triggers enables effortless inter-community connectivity.

4. Verified Identity Governance and Spam-Free Community: To safeguard the community from impostors, unauthorized access, and malicious accounts, the platform enforces mandatory 6-digit email OTP verification during registration. Furthermore, all newly registered accounts are isolated in an administrative review queue, granting access only after verification by community trustees. This two-tier gatekeeping mechanism ensures an authentic, safe, and close-knit digital environment.

5. Critical Emergency Medical Utility via Community Blood Registry: KutumbSetu acts as a life-saving tool during medical emergencies by integrating a dedicated community blood donor directory across all major blood groups. Community members can voluntarily opt-in as emergency donors, allowing families facing hospital crises to instantly filter and contact willing donors within their immediate city, bypassing lengthy hospital red tape and blood bank shortages.

6. End-to-End Campaign and Community Event Management: The platform modernizes the organization of community drives such as Blood Donation Camps, Medical Checkups, Annual General Meetings (AGM), and Youth Fests. Organizers can define headcount quotas, publish schedules, and track separate rosters for attendees and volunteers. Members receive digital registration receipts, which significantly minimizes manual paperwork and on-ground coordination chaos.

7. Moderated Community Broadcasting and Content Quality Control: Rather than allowing unmonitored group chats that quickly deteriorate into spam and misinformation, KutumbSetu utilizes an administrative moderation pipeline. Member submissions for marriage announcements, milestone birthday wishes, obituaries, and achievements must be reviewed and approved by administrators before public broadcasting, ensuring that the community feed remains respectful, relevant, and verified.

8. Culturally Responsive, Bilingual Accessibility (English & Gujarati): Recognizing that community demographics range from elderly trustees to tech-savvy youth, the application features an instant language toggle between English and Gujarati. Combined with dynamic Light/Dark mode themes and intuitive Material Design 3 tokens, the user interface remains welcoming, legible, and easy to navigate for community members of all age brackets and technological literacy levels.

5.2 Future Scope

The **KutumbSetu Project** lays a strong foundation for digital transformation within community management, genealogical tracking, and cultural preservation, but there are several opportunities for further enhancement and expansion in the future:

•**AI-Driven Automated Horoscope (Kundali) Matching:** Integrating automated Vedic astrology algorithms and *Guna Milan* calculation APIs directly into the matrimonial module will provide instant compatibility scores and AI-recommended candidate matches based on lifestyle and horoscopic alignments.

• **AI Photo Restoration and Historical Archiving**: Incorporating computer vision and deep learning models to automatically restore, colorize, and enhance ancient, damaged black-and-white family photographs uploaded to the *Vanshavali* genealogical tree, preserving ancestral history in high definition.

• **Multi-Community SaaS Platform Scalability**: Scaling the current system into a multi-tenant Software-as-a-Service (SaaS) architecture, allowing other dispersed Indian communities, cultural associations, and trusts to adopt the platform with custom branding, subcastes, and native regional languages.

• **Geo-Fencing and Proximity-Based Emergency Blood Dispatch**: Enhancing the community blood registry with real-time GPS location tracking and geofencing to broadcast urgent push notifications to registered donors who are within a 5–10 km radius of the requesting hospital during medical emergencies.

• **Integrated Payment Gateway for Trust Donations and Events**: Implementing secure digital payment interfaces (such as UPI, Razorpay, or Stripe) to streamline online contributions for community welfare funds, student educational scholarships, and digital ticket booking for matrimonial *Sammelans* and cultural events.

• **DigiLocker and Government Identity Verification**: Connecting the member onboarding workflow with government verification APIs (such as DigiLocker or Aadhaar-based KYC verification) to automate document authentication, reducing the administrative burden of manually vetting newly registered profiles.

•**Offline-First Synchronization for Rural Connectivity**: Implementing a robust local cache engine (using SQLite or Hive) with bi-directional cloud synchronization, enabling community members visiting native villages with poor network coverage to browse the directory and view family trees seamlessly offline.

5.3 Limitations

While the **KutumbSetu platform effectively addresses core community management, matrimonial privacy, and genealogical tracking requirements, the current implementation operates under the following technical and operational constraints:**

• **Dependency on Continuous Internet Connectivity:** The application relies on active internet access to communicate with the Node.js REST API and synchronize real-time updates from MongoDB. In remote rural villages with unstable or low-bandwidth mobile network coverage, users may experience latency or inability to view updated directory records and media files due to the absence of an offline-first local database cache.

• **Administrative Verification Bottleneck:** The current security workflow mandates that every newly registered account and public community post be manually vetted by an administrator before approval. While this ensures community safety and prevents spam, it creates an operational dependency on administrator availability, occasionally leading to delays in user onboarding and announcement broadcasts during high-traffic periods.

• **Reliance on User-Reported Lineage Accuracy:** The integrity and visual completeness of the interactive family tree depend directly on the authenticity of information entered by individual members. If a user provides incomplete details, misspelled ancestral names, or incorrect relational IDs, the automated graph builder may generate disconnected ancestral nodes or partial lineage branches.

• **Absence of Automated Government KYC Integration:** Although the platform enforces 6-digit email OTP verification, it does not currently integrate automated government identity verification services (such as DigiLocker or Aadhaar e-KYC). User verification remains reliant on administrator scrutiny and cross-referencing with existing physical community records.

• **Local Server Media Storage Constraints:** Uploaded media assets—such as high-resolution member profile photos, matrimonial gallery images, and community video reels—are handled via server-side storage rather than a dedicated cloud object store (such as Amazon S3) paired with a Content Delivery Network (CDN). Over time, substantial video uploads may lead to server disk space constraints and slower media delivery.

• **Heuristic Matrimonial Matchmaking:** The matrimonial recommendation engine currently operates on deterministic, rule-based criteria (filtering by opposite gender, age brackets, education, and gotra) rather than automated Vedic astrological algorithms (*Guna Milan* / Kundali matching) or AI-driven behavioral compatibility scoring.

5.4 Acknowledgment

I would like to express my sincere and heartfelt gratitude to **Mr. M.Z. Shaikh**, Principal of *Shri Bhagubhai Mafatlal Polytechnic and College of Engineering*, and **Mrs. Neeta Kadukar**, Vice Principal and Head of the Department of Information Technology, for providing us with the opportunity and resources to undertake this project titled *"KutumbSetu"*. Their continuous encouragement, academic support, and commitment to student excellence have been a great source of motivation throughout this journey.

I extend my deepest appreciation to my project mentor, **Mr. Pankaj Rathod**, for his invaluable guidance, insightful feedback, and unwavering support during every stage of the project. His expertise and constructive suggestions played a vital role in helping us refine our ideas, overcome technical challenges, and achieve our objectives successfully.

I am also grateful to all the faculty members of the Department of Information Technology for imparting the technical knowledge and professional skills that greatly contributed to the successful completion of this project. Their teaching and mentorship have laid a strong foundation for our learning and development.

Lastly, I would like to acknowledge that this project was completed entirely by me and my team with dedication, collaboration, and consistent effort. The experience of working on this project has been both enriching and rewarding, allowing us to apply our classroom learning to real-world problem-solving.

5.5 Conclusion

The **KutumbSetu Project** represents a significant step forward in the digital modernization of community management, genealogical heritage preservation, and matrimonial networking. By shifting from fragmented manual methods—such as printed member directories and unmonitored social messaging groups—to a centralized, secure, and interactive mobile platform, the community can overcome long-standing challenges related to record obsolescence, privacy vulnerabilities in biodata circulation, and geographical disconnection among dispersed members. This project not only streamlines member discovery and preserves multi-generational family trees, but also empowers community administrators with transparent governance tools, a vetted social broadcasting pipeline, and a life-saving emergency blood donor registry. With continued enhancements and future expansions, the KutumbSetu Project has the potential to become a model digital solution that drives unity, cultural preservation, accountability, and generational connectivity within the community and beyond.

5.6 References

[1] Flutter, Flutter Documentation: Build Apps for Any Screen. [Online].

Available: https://docs.flutter.dev/

[2] Dart, Dart Programming Language Guide & Documentation. [Online].

Available: https://dart.dev/guides

[3] Riverpod, Riverpod Reactive State Management Documentation. [Online].

Available: https://riverpod.dev/docs/introduction/getting_started

[4] Node.js, Introduction to Node.js – Node.js Documentation. [Online].

Available: https://nodejs.org/en/learn/getting-started/introduction-to-nodejs

[5] Express.js, Express – Fast, Unopinionated, Minimalist Web Framework for Node.js. [Online].

Available: https://expressjs.com/

[6] MongoDB, MongoDB Manual and Database Documentation. [Online].

Available: https://www.mongodb.com/docs/

[7] Mongoose, Mongoose ODM Documentation: Elegant MongoDB Object Modeling for Node.js. [Online].

Available: https://mongoosejs.com/docs/

[8] Nodemailer, Nodemailer: Send E-mails with Node.js. [Online].

Available: https://nodemailer.com/

[9] Flutter Community, GoRouter – Declarative Routing Package for Flutter. [Online].

Available: https://pub.dev/packages/go_router

[10] Postman, Postman Learning Center: API Platform Overview. [Online].

Available: https://learning.postman.com/docs/introduction/overview/

[11] Google, Material Design 3 Guidelines & Design Tokens. [Online].

Available: https://m3.material.io/

[12] Git, Git Distributed Version Control Documentation. [Online].

Available: https://git-scm.com/doc